

A two stage lightweight approach for intrusion detection in Internet of Things

Zeyu Li, Wenbin Yao *

School of Computer Science (National Pilot Software Engineering School), Beijing University of Posts and Telecommunications, Beijing 100876, China

ARTICLE INFO

Keywords:

Intrusion detection
Internet of Things
Self-supervised learning
Self-knowledge distillation

ABSTRACT

The prevalence of the Internet of Things (IoT) in people's life has led to a significant rise in the amounts of breaches against different IoT devices. Given that IoT devices are typically installed in settings with constrained resources and limited processing power, and malicious traffic samples are difficult to collect, resulting in traditional deep learning algorithms that do not handle intrusion detection in IoT network well. In this work, we present a two-stage lightweight intrusion detection model based on self-supervised contrastive learning and self-knowledge distillation to reduce model's over-reliance on labels, optimize the model's capacity for generalization and significantly increase the IoT network's intrusion detection speed while lowering the model's complexity. The representations of the network traffic are extracted in the first step using self-supervised contrastive learning, and the representations learnt by the teacher model are transferred to the student model in the second stage using self-knowledge distillation. On the benchmark datasets KDD CUP99, NSL-KDD, UNSW-NB15, CIC IDS2017 and BoT-IoT, we carried out in-depth studies, and the experimental results demonstrate that our proposed model outperforms recent state-of-the-art models in addition to achieving superior intrusion detection capabilities. The code will be released at <https://github.com/RicardoMLi/CL-SKD>.

1. Introduction

The Internet of Things, or IoT, is a network that grows outward from the Internet, it is a continuation of the Internet, and its core and foundation is still the Internet. The IoT is a network which uses sensing devices to link any object to the Internet. It then exchanges data and communicates using established protocols to accomplish intelligent object identification, placement, tracking, monitoring, and management. The rise of IoT technology has changed the new direction of the information world and is considered the third wave of information development. Nowadays, IoT technologies are quietly changing our lifestyles and providing various conveniences to our lives, such as smart cities (Kim et al., 2017), healthcare (Aghdam et al., 2021), smart home (Zielonka et al., 2020) and wearable devices (Dian et al., 2020). However, due to the lack of cybersecurity knowledge among IoT device owners, hackers attack on cyber-physical devices such as wearable devices, medical pacemakers, autonomous vehicles or expensive industrial processes controlled by connected devices, which can result in the theft of personal or corporate private data, causing great economic loss and sometimes even serious life safety incidents (Alaba et al., 2017). Fig. 1

illustrates the overall structure of the IoT.

Although cybersecurity experts have taken many efforts to improve IoT security, including encryption of information transmitted over the network, updating firmware regularly, using strong passwords and security keys (Roy et al., 2022). However, even with the above countermeasures, IoT devices are still susceptible to many types of cyberattacks because of their diversity. The industry is now concentrating its attention on ways to lessen the damage caused by IoT device incursions, intrusion detection system referred as a prominent component of cyber security, has become an important means of network attack detection. Intrusion detection systems may be classified as either Misuse based Intrusion Detection System (MIDS) (Agarwal & Joshi, 2001) or Anomaly based Intrusion Detection System (AIDS) (Chandola et al., 2009) depending on the detection methodologies used. However, because MIDSs rely heavily on the existing known signature database, it is challenging to identify zero-day attacks and they cannot be used to identify unknown attacks (Abdel-Basset et al., 2020). When AIDSs detect the system behavior isn't typical, they identify it as aberrant system behavior and send out an alarm to the administrator. Attack detection in IoT networks has been facilitated by the widespread deployment of a

* Corresponding author.

E-mail addresses: ricardolee@bupt.edu.cn (Z. Li), yaowenbin@bupt.edu.cn (W. Yao).

<https://doi.org/10.1016/j.eswa.2024.124965>

Received 16 October 2023; Received in revised form 30 July 2024; Accepted 31 July 2024

Available online 5 August 2024

0957-4174/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

large variety of intrusion detection systems based on machine learning and deep learning in recent years (Da Costa et al., 2019; Murali & Jamalipour, 2019; Zhou et al., 2021), however, there are still many challenges in detecting anomalous traffic in IoT network. On the one hand, the majority of IoT network nodes are frequently placed in devices with restricted power and computational capabilities, as well as in unfavorable communication conditions with limited storage capacity. On the other hand, obtaining labeled abnormal traffic is expensive and time-consuming, in order to ascertain whether the network traffic represents a novel form of attack, cyber security professionals must be consulted (Wang, Li, Wang, & Li, 2021). In addition, different protocol stacks and protocol standards are applied in IoT network, and these requirements make it necessary for intrusion detection systems to design security mechanisms according to different protocol standards and stacks (Roy et al., 2022). Therefore, for an IoT network, an effective intrusion detection system must be lightweight, real-time, and unsupervised. Nevertheless, most of the existing intrusion detection systems only meet one of the three characteristics, and few intrusion detection systems meet all three characteristics. To address this challenge, self-supervised learning and knowledge distillation have emerged as key techniques. The advantage of self-supervised learning lies in its ability to eliminate the need for labeled data, significantly reducing the cost and time associated with data preparation. On the other hand, knowledge distillation effectively compresses the model size, reducing the number of parameters and enhancing the system's computational efficiency and response speed. The combination of these two technologies holds the promise of developing an advanced IoT intrusion detection system that is lightweight, real-time, and capable of operating in an unsupervised manner.

In this study, we present a two-stage lightweight intrusion detection model based on self-supervised contrastive learning and self-knowledge distillation using the ideas of self-supervised learning (Koohpayegani et al., 2021) and self-knowledge distillation (Fang et al., 2021) called CL-SKD for short, so as to construct an IoT intrusion detection system with the characteristic of lightweight, real-time and unsupervised. In our paper, we divide intrusion detection into two phases, in the first phase, to identify the essential feature representations of network traffic without labels, we leverage the ability of self-supervised contrastive learning to train a model in the absence of labeled data. Then we use self-knowledge distillation to transfer the feature representations learned by convolutional neural network to depthwise separable convolutional neural network in the second phase, and through knowledge

distillation, we can retain the feature extraction capability of the teacher network to the maximum extent possible. In this paper, by using KDD CUP99, NSL-KDD, UNSW-NB15, CIC IDS2017 and BoT-IoT as IoT intrusion detection datasets, we will carry out binary and multi classification experiments to fully compare the recent state-of-the-art models with proposed CL-SKD model, so as to prove the powerful generalization ability and excellent capacity for anomaly detection of the proposed model in our paper.

In summary, the following advances in the field of intrusion detection are brought about by the work presented in our study.

- 1) We present an efficient and lightweight intrusion detection model for IoT network based on self-supervised contrastive learning and self-knowledge distillation, it uses knowledge distillation and depthwise separable convolutional neural network to condense the model's size without sacrificing its ability to identify anomalies, while avoiding the model's over-reliance on labeled data.
- 2) Improve the self-supervised contrastive learning algorithm, optimize the cost function of self-supervised contrastive learning, and improve the model's capacity for both representation learning and generalization.
- 3) Simplify the cost function of self-knowledge distillation, reduce calculation workload of the model cost function to improve the model's speed of intrusion detection.
- 4) By using KDD CUP99, NSL-KDD, UNSW-NB15, CIC IDS2017 and BoT-IoT as IoT intrusion detection datasets, we conduct extensive experiments using multiple evaluation metrics and comparing the latest state-of-the-art models with proposed CL-SKD model.

Here is the arrangement for the remaining portion of the paper. Section 2 describes the intrusion detection related techniques for IoT network. In Section 3, the intrusion detection model based on self-supervised contrastive learning and self-knowledge distillation is introduced. The datasets utilized in this work and their pre-processing methods are briefly introduced in Section 4. The experimental simulation settings, model implementation specifics, and model assessment metrics are presented in Section 5. In Section 6, we verify the effectiveness of the proposed CL-SKD model using self-supervised contrastive learning and self-knowledge distillation technologies through a plenty of experiments. In Section 7, some pertinent findings are presented and further study is proposed.

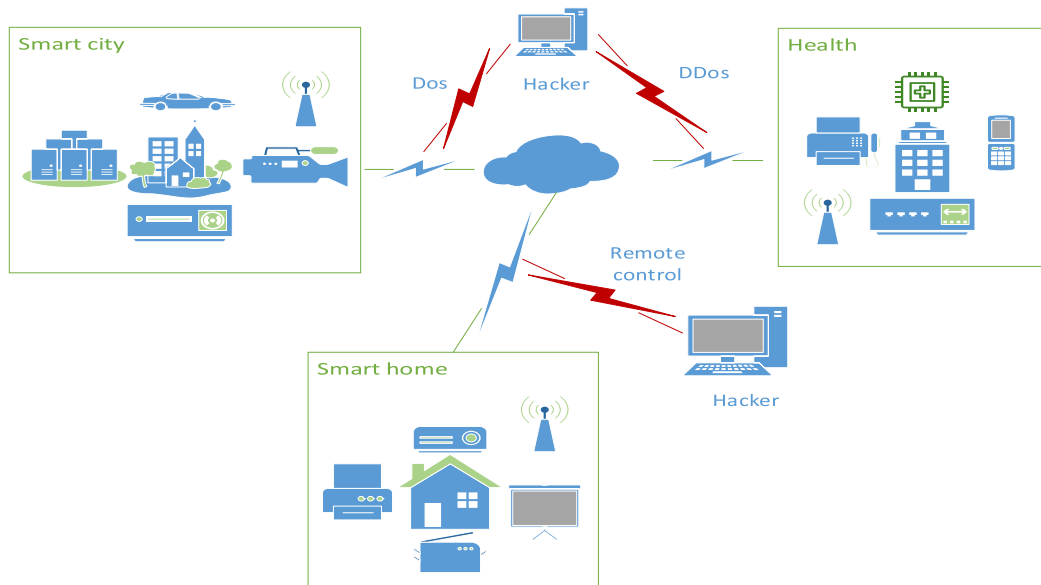


Fig. 1. Structure of the IoT.

2. Related works

This section will cover a few intrusion detection algorithms based on machine learning and deep learning for IoT networks.

2.1. Intrusion detection solutions based on conventional machine learning

Because of their benefits of easy implementation, high generalization ability, solid mathematical and theoretical foundation as well as ease of understanding and interpretation, machine learning algorithms including K nearest neighbor (KNN), decision tree (DT), support vector machine (SVM), and naïve bayes (NB) are commonly employed in the intrusion detection field (Ding et al., 2022, Guezzaz et al., 2021, Safaldin et al., 2021, Gu & Lu, 2021).

Gad et al. (2021) utilized chi-square test to determine the optimal feature subset of the ToN-IoT dataset, they then used the synthetic minority over-sampling technique to create rare attack traffic and reduce data imbalance, and finally applied the extreme gradient boosting (XGBoost) classifier to detect attack traffic in the IoT network. In contrast to machine learning techniques, the XGBoost algorithm has better detection capability, the study only tested this model on the ToN-IoT dataset and did not extend it to other IoT intrusion detection datasets, leaving its generalization capability in question. Fraihat et al. (2023) proposed an intrusion detection model based on NetFlow features and an improved arithmetic optimization algorithm (AOA). The authors significantly reduced the number of features by selecting the optimal ones through the improved AOA. Experimental results showed that the proposed NIDS achieved 99 % accuracy in binary classification tasks and 98 % accuracy in multi-class classification tasks. Nimbalkar and Kshirsagar (2021) proposed to use Information Gain and Gain Ratio to select the top 50 % features of IoT-BoT and KDD CUP99 datasets for IoT anomaly detection, where 16 and 19 characteristics, respectively, may achieve 99.9993 % and 99.9992 % classification accuracy for the KDD CUP99 and IoT-BoT datasets. Gaber et al. (2023) proposed a new intrusion detection model based on Industrial IoT network traffic. The authors used particle swarm optimization and bat algorithm for feature selection and employed a random forest classifier to classify malicious behavior. Experimental results on the WUSTL-IIOT-2021 dataset demonstrated that the proposed model performed excellently in both binary classification and multi-class classification tasks. Jan et al. (2019) presented a SVM algorithm-based intrusion detection model with full consideration of energy use and processing capabilities of IoT devices. To further reduce the computational cost, the model only considered the attribute of packet transmission rate and extracted only 2 to 3 features from this attribute, and the experimental findings showed that the proposed model achieved classification accuracy of 98.35 %, and the model training time was only 16.3 s.

Shallow learning is what most machine learning algorithms do, which cannot handle classification of large-scale and high-dimensional data well. For instance, the DT algorithm relies too much on training data, which leads to overfitting easily and is more difficult to handle the missing data (Singh & Gupta, 2014). The adaptive boosting (AdaBoost) algorithm is too sensitive to abnormal samples, besides, there are dependencies between weak classifiers, which makes it difficult to train data in parallel and limits the training efficiency of the AdaBoost algorithm (Ge et al., 2010). Therefore, with the dynamic growth of the datasets, a large number of researchers apply deep learning-based algorithms to solve the vulnerabilities and security issues in the IoT network.

2.2. Intrusion detection solutions based on deep learning

As computational power and computer hardware have advanced, deep learning (LeCun, Bengio, & Hinton, 2015) has gradually started to lead the trend in various fields of computing (computer vision (Voulodimos et al., 2018), natural language processing (Li, 2018), and speech

recognition (Zhang et al., 2018), etc.), intrusion detection is no exception.

In recent years, an increasing number of academics are attempting to apply deep learning algorithms to address the intrusion detection problem. For example, Lazzarini, Tianfield, and Charissis (2023) proposed a novel IoT intrusion detection method based on a stacked ensemble of deep learning models, including multilayer perceptron (MLP), deep neural network (DNN), convolutional neural network (CNN), and long short-term memory network (LSTM). Experimental results on the ToN-IoT, CICIDS2017, and SWaT datasets demonstrated that the proposed model exhibited high accuracy and low false alarm rates across all datasets. Telikani et al. (2022) presented a hybrid intrusion detection model based on stacked autoencoders and convolutional neural networks, introducing a novel cost-dependent loss function. Experimental results on the ToN-IoT and UNSW-NB15 datasets demonstrated that the proposed model achieved good intrusion detection performance. Zhao et al. (2021) proposed an intrusion detection method based on a lightweight deep neural network. During the data preprocessing stage, the principal component analysis algorithm was used to reduce the dimensionality of the features. Additionally, the classifier employed an expanded and compressed structure, an inverted residual structure, and channel shuffle operations to achieve effective feature extraction with low computational cost. Roy et al. (2022) proposed a new machine learning-based intrusion detection model that identified the most important features through a series of optimizations, including the removal of multicollinearity, sampling, and dimensionality reduction. The experimental results on CIC IDS2017 and NSL-KDD datasets demonstrated that the model has a high detection rate and a low false alarm rate. Khanday, Fatima, and Rakesh (2023) proposed a lightweight intrusion detection model for DDoS attacks. The researchers introduced a novel data preprocessing method and used machine learning and deep learning to detect anomalies in IoT network traffic data. Experimental results on TON-IoT and UNSW-NB15 datasets indicated that the model performed well in detecting DDoS attacks. To address the issue of class imbalance in intrusion detection, Telikani et al. (2023) proposed an intrusion detection model based on multi-task learning. The model used an autoencoder for feature extraction and enhanced the SVM classifier through a cost-sensitive learning strategy, improving its ability to recognize minority class intrusions. Experiments on the UNSW-NB15 and BoT-IoT datasets showed that the proposed model outperformed other methods. To accommodate the changing nature of network traffic behaviour over time, dos Santos et al. (2022) introduced an intrusion detection model based on reinforcement learning. The model updated itself through transfer learning and a sliding window mechanism, reducing the need for computational resources and manual intervention.

Different from existing intrusion detection models, we take the IoT network nodes' limited power, processing, communication and storage capabilities into full consideration, we also take the model generalization capability and the difficulty of collecting abnormal network traffic into account. Then we utilized self-supervised contrastive learning and self-knowledge distillation to propose a two-stage lightweight intrusion detection model. We conduct extensive experiments on the IoT intrusion detection datasets KDD CUP99, NSL-KDD, UNSW-NB15, CIC IDS2017 and BoT-IoT for binary and multi classification, and utilize evaluation metrics such as accuracy, precision, F-measure and receiver operating characteristic (ROC) curve to assess the model's performance, which are more scientific and comprehensive for the evaluation of the CL-SKD model.

3. Methodology

The CL-SKD model presented in this study has two phases, as seen in Fig. 2. We learn the feature representations of the essential components of network traffic by self-supervised contrastive learning during the first phase. In the second stage, the feature representations learnt from the

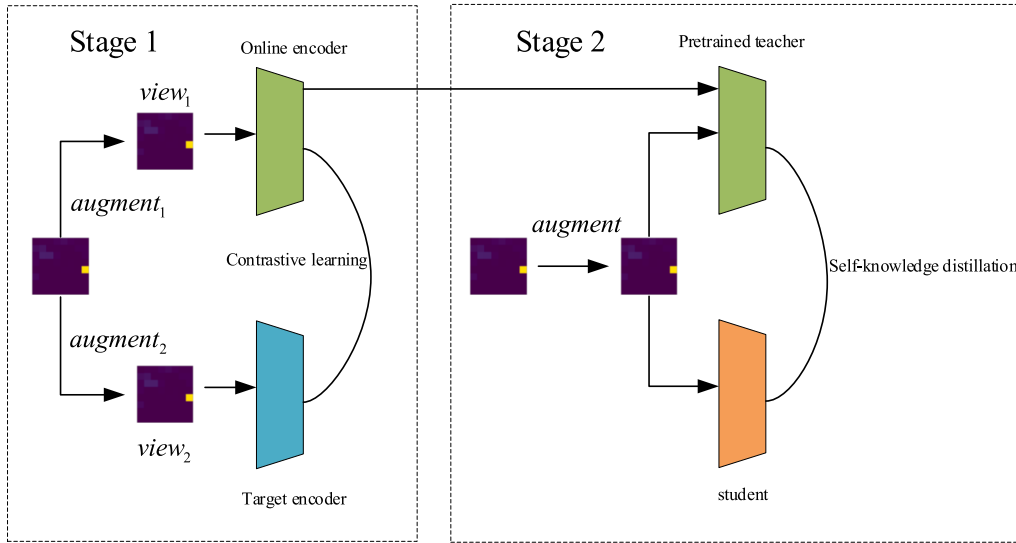


Fig. 2. Overall architecture of CL-SKD model.

convolutional neural network are transferred to the depthwise separable convolutional neural network via self-knowledge distillation. We will then carefully discuss the specific implementation steps of these two stages and the implementation of depthwise separable convolution.

3.1. Self-supervised contrastive learning

In recent years, a number of innovative self-supervised contrastive learning algorithms have shown impressive results in computer vision (image classification, image segmentation, and object recognition, etc.), such as SimCLR (Chen et al., 2020), MoCo (He et al., 2020), BYOL (Grill et al., 2020), DINO (Caron et al., 2021) and so on. Both normal and abnormal network traffic follow certain patterns, such as following the same protocols, datagram size limits, etc., and such patterns can be mined in a dataset by self-supervised contrastive learning algorithms and transferred to other intrusion detection related datasets, thereby improving the generalization and accuracy of detection. Moreover, self-supervised contrastive learning algorithms have unique advantages. They can automatically extract useful features from data through contrastive learning without relying on labeled data. This capability makes them particularly suitable for network traffic analysis, as network data is often vast and complex, making manual labeling costly and challenging to cover all possible attack patterns. By using self-supervised contrastive learning algorithms, hidden features in both normal and anomalous traffic can be automatically captured, enabling the development of effective detection models.

Our previous work (Wang, Li, Wang, & Li, 2021) is based on BYOL algorithm, a self-supervised contrastive learning algorithm (Instead of comparing network traffic image instances, the BYOL algorithm learns rich feature representations by pulling the views of network traffic images under two different data augmentations closer). Although BYOL is a simple and elegant algorithm, inspired by MSF (Koohpayegani et al., 2021), we try to pull the views of the same network traffic image under different data augmentations closer, but furthermore draw its closest neighbours in the feature embedding space closer, as those neighbours can belong to the same class as the network traffic in question. Therefore, we cannot discriminate the nearest neighbours as negative examples and push the nearest neighbours away from it in the feature embedding space. Similar to the BYOL algorithm, we also maintain two asymmetric neural networks, named Online Network and Target Network in MSF framework. Both Online Network and Target Network consist of a feature encoder which is used to obtain the representations of the network traffic, it usually consists of CNN (In self-supervised

learning stage, the reason why we use CNN rather than depthwise separable convolution is that conventional convolutional neural networks have more powerful feature extraction capability compared to depthwise separable convolution. We also want to keep a network with good feature extraction capabilities and transfer its knowledge to a smaller network), and a feature projector which is used to transfer the representation space to projection space for dimension reduction, it usually consists of two dense layer and a batch normalization layer, but Online Network adds a feature predictor after the feature projector which consists of two dense layer and a batch normalization layer. Assuming that Online Encoder and Target Encoder are represented by f_θ and f_ζ , Online Projector and Target Projector are denoted by g_θ and g_ζ , respectively, and Online Predictor is denoted by q_θ , where θ and ζ represent the weights of Online Network and Target Network, respectively. To store the nearest neighbours under different augmented views, we maintain a first-in-first-out memory queue, which is composed of feature embeddings encoded by Target network, where we put the newest batches encoded by Target network into the queue and put the oldest batches out of the queue during training. The specific training process of self-supervised contrastive learning is shown in Fig. 3.

Given an unlabelled network traffic x , the shape of x is $14 \times 14 \times 1$ after preprocessing, so we do two distinct sets of data augmentation processes T_1 and T_2 on x to obtain two different views $T_1(x)$ and $T_2(x)$ of x . Subsequently, $T_1(x)$ and $T_2(x)$ are fed into the feature encoders f_θ and f_ζ to obtain two different sets of feature embeddings y_1 and y_2 , i.e., $y_1 = f_\theta(T_1(x))$, $y_2 = f_\zeta(T_2(x))$. Then the two different sets of feature embeddings y_1 and y_2 are fed into the feature projectors g_θ and g_ζ to obtain two different sets of feature projections z_1 and z_2 , i.e., $z_1 = g_\theta(y_1)$, $z_2 = g_\zeta(y_2)$. Then z_1 is fed into the feature predictor to get the feature query q , i.e., $q = q_\theta(z_1)$, followed by a l_2 regularization for q and z_2 , respectively, i.e.

$$\bar{q} = \frac{q_\theta(z_1)}{\|q_\theta(z_1)\|_2} \quad (1)$$

$$\bar{z}_2 = \frac{g_\zeta(y_2)}{\|g_\zeta(y_2)\|_2} \quad (2)$$

Then we add $\bar{z}_2$ to the memory queue first, and find the K nearest neighbours of $\bar{z}_2$ in the memory queue to get a set of feature embeddings denoted as $\{n_i\}_{i=1}^K$. We want the average distance between $\bar{q}$ and the feature embeddings in $\{n_i\}_{i=1}^K$ to be the smallest, so we can minimize the following loss function.

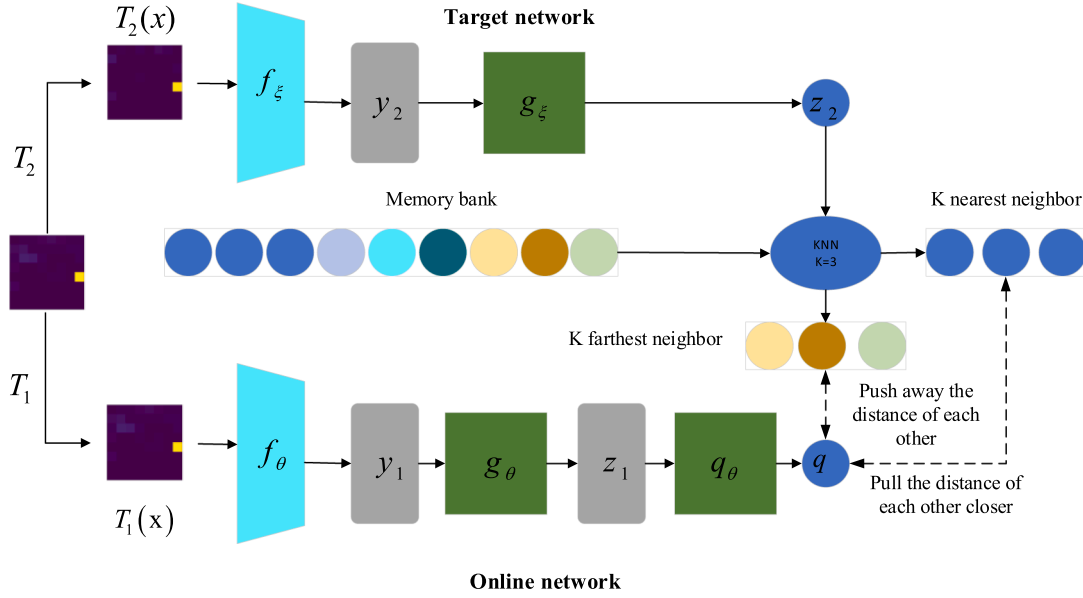


Fig. 3. Self-supervised contrastive learning training process.

$$L_{nearest} = \frac{1}{K} \sum_{i=1}^K \text{dist}(\bar{q}, n_i) \quad (3)$$

where K is the number of nearest neighbours of $\bar{z}_2$, $\text{dist}(p, q)$ represents the distance measure between two feature embeddings, we can use mean square error loss, i.e., $\text{dist}(p, q) = \|p - q\|_2^2$ as the distance between two feature embeddings.

Different from (Koohpayegani et al., 2021), as we all know, the parameters of a neural network are usually initialized randomly, we believe that in the early stages of training, the feature encoder cannot obtain high-quality feature embeddings of the network data, so the feature embeddings obtained by finding the K nearest neighbours of $\bar{z}_2$ do not represent the same class as $\bar{z}_2$. Therefore, we think that the feature encoder can push the distance of feature embeddings $\{f_i\}_{i=1}^K$ obtained from the K farthest neighbours of $\bar{z}_2$ away from $\bar{z}_2$, and we hope that the distances between $\bar{z}_2$ and $\{f_i\}_{i=1}^K$ become larger so that we can avoid the feature extractor from falling into a local optimum. Therefore, we add the average distance $L_{farthest}$ between feature embeddings $\{f_i\}_{i=1}^K$ and $\bar{q}$ as a component of the cost function (Although this move adds a bit of computational load, we believe it is a trade-off between performance and computation.), while the weight coefficient accounted for $L_{farthest}$ is gradually reduced to 0 to save the computational cost after the feature encoder being able to obtain good feature embeddings of the network traffic as the model continuously trained, so the loss function of the improved self-supervised contrastive learning can be calculated by following equation.

$$L = L_{nearest} - \alpha * L_{farthest} = \frac{1}{K} \left(\sum_{i=1}^K \text{dist}(\bar{q}, n_i) - \alpha * \sum_{i=1}^K \text{dist}(\bar{q}, f_i) \right) \quad (4)$$

where α is the weight coefficient, and we use a linear reduction to control the magnitude of α , the value of α at the t -th epoch can be obtained using the following formula.

$$\alpha = \begin{cases} (T - t)/T, & \text{if } t < T/2 \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

where T and t are the total number of epochs and the current number of the epoch. After a lot of experiments, we found that when $t = T/2$, better feature embeddings of the network traffic may be obtained via the

feature encoder, so we set α to 0 in the subsequent model training.

In Online Network, the feature encoder f_θ , feature projector g_θ and feature predictor q_θ update their parameters by back propagation algorithm, while in Target Network the feature encoder f_ζ and feature projector g_ζ update their parameters by momentum, i.e.

$$\zeta \leftarrow \eta * \zeta + (1 - \eta) * \theta \quad (6)$$

where $\eta \in [0, 1]$, it is a hyperparameter that regulates how much the Target network's parameters rely on the Online network's parameters. Algorithm 1 illustrates the precise procedures of self-supervised contrastive learning.

Algorithm 1: Pseudocode of SSL in a Pytorch-like style

Input: Input dataset: X , online network: f, q , target network: f, k , memory queue to store the feature embeddings of target network: $queue$, momentum coefficient: η , size of memory queue: men_size , weight coefficient between $L_{farthest}$ and $L_{nearest}$: α .

Output: the best model of online network: f, q

```

1:  $f.k.params = f.q.params$ 
2:  $f.k.params.require\_grad = False$ 
3: for  $x$  in  $X$ :
4:    $x\_q = \text{augment\_q}(x)$ 
5:    $x\_k = \text{augment\_k}(x)$ 
6:    $query = f.q(x\_q).normalize()$ 
7:    $target = f.k(x\_k).normalize()$ 
8:    $\text{enqueue}(queue, target)$ 
9:    $\text{dequeue}(queue)$ 
10:   $targets = queue.copy().detach()$ 
11:  # get the distances between current target and the feature embeddings in memory queue
12:   $dist\_k = \text{bmm}(target, targets.view(-1, men\_size))$ 
13:  # find the top  $k$  nearest and top  $k$  farthest embeddings in memory queue

```

copy online network weights to target network
freeze the weights of target network
data augmentation for x
another data augmentation for x
forward through the online network
forward through the target network
enqueue the current minibatch
dequeue the earliest minibatch
get the full feature embeddings in memory queue

(continued on next page)

(continued)

Algorithm 1: Pseudocode of SSL in a Pytorch-like style

```

14: nearest_k = dist_k.topk(largest = False)
15: farthest_k = dist_k.topk(largest = True)
16: #calculate the distance between top k
   nearest embeddings and query
17: nearest_dis = distance(query, nearest_k)
18: # calculate the distance between top k
   farthest embeddings and query
20: farthest_dis = distance(query, farthest_k)
21: loss = nearest_dis - alpha* farthest_dis
22: loss.update()
23: update(f.q.params)                # update online network
24: f_k.params = (1-eta)*f.q.params + # momentum update target
   eta*f_k.params                    network
25: save_weights(f.q)                # save the weights of f.q to local

```

bmm: batch matrix multiplication

3.2. Self-knowledge distillation

Knowledge distillation is a popular method for model compression, in contrast to quantization and pruning, which is first proposed by Hinton on image classification tasks (Hinton, Vinyals, & Dean, 2015). He built a tiny lightweight model and used the large model's supervised knowledge that had been pre-trained on a larger dataset to train the smaller model, so that the small model was able to obtain better performance. Commonly, we refer to the large model as the teacher model and the small model as the student model. Deploying large models on IoT network is not feasible because of low power, computation and poor communication environment. Consequently, the knowledge distillation process is necessary to transfer the information from the large model to the tiny model, and then we must deploy the tiny model for intrusion detection in the resource-constrained IoT network. Knowledge distillation can significantly compress the model size while maintaining its performance and accuracy. Through this process, the smaller model can inherit the intelligence and complexity of the larger model, but with greatly reduced computational and storage requirements, making it suitable for the resource constraints of IoT devices.

However, in order to inform the training phase of classic knowledge distillation algorithms, label information of network traffic is often required, and it is universally acknowledged that anomalous data in IoT devices is difficult to obtain, so it is worth considering how to construct a supervisory signal rather than label information to guide the training process of knowledge distillation. Inspired by (Fang et al., 2021), we can construct a first-in-first-out memory queue where we put the newest batches into the queue and put the oldest batches out of the queue during training, and the teacher model which is pre-trained through self-

supervised contrastive learning, will receive the feature embeddings acquired by delivering the unlabelled set of network traffic, and these will be placed to the memory queue, while the student model will get the unlabelled set of network traffic to obtain another set of feature embeddings too, and we can limit the distance between the two feature embedding sets and the feature embeddings in the memory queue in order to provide the supervised signal that will direct the knowledge distillation training process. We refer to the teacher model as f_θ^T and the student model as f_ζ^S , where θ and ζ are the weights of the teacher model and the student model. The common architecture of teacher and student model is a feature encoder which consists of three convolution layers followed by batch normalization layer and rectified linear unit (ReLU) and a feature projector which consists of two dense layer and a batch normalization layer as well as ReLU. While teacher model has an extra feature predictor which consists of two dense layer and a batch normalization layer as well as ReLU. Fig. 4 illustrates the specific training process of self-knowledge distillation.

Given an unlabelled network traffic image x_i , we first perform data augmentation operation on x_i to obtain x'_i , and subsequently feed x'_i into the teacher model f_θ^T and the student model f_ζ^S to get two sets of feature embeddings y_i^T and y_i^S , respectively, i.e., $y_i^T = f_\theta^T(x'_i)$, $y_i^S = f_\zeta^S(x'_i)$, followed by a l_2 regularization of y_i^T and y_i^S , respectively, i.e.

$$\bar{y}_i^T = \frac{f_\theta^T(x')}{\|f_\theta^T(x')\|_2} \quad (7)$$

$$\bar{y}_i^S = \frac{f_\zeta^S(x')}{\|f_\zeta^S(x')\|_2} \quad (8)$$

Then we add $\bar{y}_i^T$ to the memory queue, which is assumed to be denoted as $Q = \{q_1, q_2, q_3, \dots, q_K\}$, where q_j is the feature embedding obtained from the teacher model. Then we calculate the distance $P^T(x_i, \theta, Q)$ between the feature embedding $\bar{y}_i^T$ of the teacher model and all the feature embeddings in the memory queue Q as follows.

$$P^T(x_i, \theta, Q) = [p_1^T, p_2^T, \dots, p_K^T], p_j^T = \frac{\exp(\bar{y}_i^T \bullet q_j / \tau^T)}{\sum_{q \in Q} \exp(\bar{y}_i^T \bullet q / \tau^T)} \quad (9)$$

where τ^T is the temperature parameter of the teacher network, $(\bullet)$ means the inner product between the two feature embeddings, and K is the length of the memory queue.

Similarly, the distance $P^S(x_i, \zeta, Q)$ between the feature embedding $\bar{y}_i^S$ of the student network and all the feature embeddings in the memory queue Q can be obtained from above formula as follows.

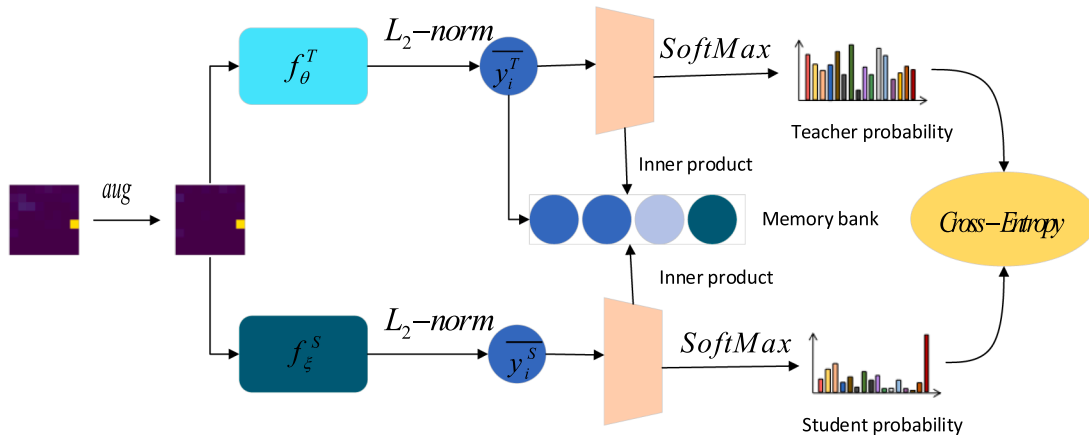


Fig. 4. Self-knowledge distillation training process.

$$P^S(x_i, \theta, Q) = [p_1^S, p_2^S, \dots, p_K^S], p_j^S = \frac{\exp(\bar{y}_i^S \bullet q_j / \tau^S)}{\sum_{q \in Q} \exp(\bar{y}_i^S \bullet q / \tau^S)} \quad (10)$$

where τ^S is the temperature parameter of the student network.

We want the distribution of $P^T(x_i, \theta, Q)$ and $P^S(x_i, \zeta, Q)$ to be similar, so the cross-entropy loss may be used as the self-knowledge distillation cost function, i.e.

$$L = \sum_i^N -P^T(x_i, \theta, Q) \bullet \log P^S(x_i, \zeta, Q) \\ = - \sum_i^N \sum_j^K \frac{\exp(\bar{y}_i^T \bullet q_j / \tau^T)}{\sum_{q \in Q} \exp(\bar{y}_i^T \bullet q / \tau^T)} \bullet \log \frac{\exp(\bar{y}_i^S \bullet q_j / \tau^S)}{\sum_{q \in Q} \exp(\bar{y}_i^S \bullet q / \tau^S)} \quad (11)$$

where N represents the total number of samples, K is the length of memory queue.

Usually, in order to keep the memory queue consistent, we will set the K to a large value (such as 65536) so that the model can observe more negative samples and thus achieving better performance. Unlike (Fang et al., 2021), we adequately account for the constrained processing capability of IoT devices, the larger value of K is set, the more the computation of the model will surge. While most of the feature embeddings maintained in the memory queue Q are random and independent of the target feature embedding $\bar{y}_i^T$, so most of the elements are very small when calculating $P^T(x_i, \theta, Q)$ and $P^S(x_i, \zeta, Q)$, thus leads to the fact that when calculating the cross-entropy loss of $P^T(x_i, \theta, Q)$ and $P^S(x_i, \zeta, Q)$, most of the elements contribute very little to the overall loss, which is basically negligible, and it is a waste of computational power to calculate the loss of these elements. Therefore, we can find the k nearest neighbours of feature embedding $\bar{y}_i^T$ in the memory queue Q , and then compute the distances between them, which can be indicated as $P_T(x_i, \theta, Q^T)$, where $Q^T = \{q_i^T\}_{i=1}^k$. Similarly, we can find the k nearest neighbours of feature embedding $\bar{y}_i^S$ in the memory queue Q , and then compute the distances between them, which can be denoted as $P_S(x_i, \theta, Q^S)$, where $Q^S = \{q_i^S\}_{i=1}^k$, and $k \ll K$. Then we can calculate the cross-entropy loss of $P_T(x_i, \theta, Q^T)$ and $P_S(x_i, \theta, Q^S)$ as the loss function for self-knowledge distillation, i.e.

$$L = \sum_i^N -P_T(x_i, \theta, Q^T) \bullet \log P_S(x_i, \zeta, Q^S) \\ = - \sum_i^N \sum_j^k \frac{\exp(\bar{y}_i^T \bullet q_j / \tau^T)}{\sum_{q \in Q^T} \exp(\bar{y}_i^T \bullet q / \tau^T)} \bullet \log \frac{\exp(\bar{y}_i^S \bullet q_j / \tau^S)}{\sum_{q \in Q^S} \exp(\bar{y}_i^S \bullet q / \tau^S)} \quad (12)$$

where k is the number of nearest neighbours of feature embedding $\bar{y}_i^T$.

After getting the loss function of self-knowledge distillation, we may use the back propagation approach to update the student model's weights, and finally the obtained student model is able to learn excellent feature representations of network traffic, which can be transferred to other intrusion detection datasets for anomaly detection. The specific steps of self-knowledge distillation are shown in Algorithm 2.

Algorithm 2: Pseudocode of SKD in a Pytorch-like style

Input: Input dataset: X , teacher network: f_t , student network: f_s , memory queue to store the feature embeddings of teacher network: $queue$, the size of memory queue: K , top k nearest neighbor: k , temperatures for teacher network: $temp_t$, temperatures for student network: $temp_s$.
Output: the best model of student network: f_s

(continued on next column)

(continued)

Algorithm 2: Pseudocode of SKD in a Pytorch-like style

```

1: f_t.params.require_grad = False           # freeze the weights of teacher
                                                network
2: for image in X:
3:   image = augment(image)                  # apply data augmentation to
                                                image
4:   t_emb = f_t(image).normalize()           # forward through teacher
                                                network
5:   s_emb = f_s(image).normalize()           # forward through student
                                                network
6:   enqueue(queue, t_emb)                   # enqueue the current
                                                minibatch
7:   embeddings = queue.copy().detach()       # get the full feature
                                                embeddings in memory queue

8:   # get the distances between teacher and
   feature embeddings in memory queue
9:   dist_t = bmm(t_emb, embeddings.view
   (-1, K))
10:  # get the distances between student and
   feature embedding in memory queue
11:  dist_s = bmm(s_emb, embeddings.view
   (-1, K))
12:  # find the top k nearest embeddings in
   dist_t and dist_s
13:  topk_t = dist_t.topk(k)
14:  topk_s = dist_s.topk(k)
15:  # apply temperatures for soft-labels
16:  topk_s = topk_s / temp_s
17:  topk_t = softmax(topk_t / temp_t)
19:  # calculate loss using log_softmax for
   stable computation
20:  loss = -bmm(topk_t, log_softmax(topk_s))
21:  # dequeue the earliest minibatch
22:  dequeue(queue)
23:  loss.backward()
24:  update(f_s.params)                       # update student network
bmm: batch matrix multiplication

```

3.3. Depthwise separable convolution

In order to accomplish real-time intrusion detection, we will reduce the model complexity and computation in two aspects: (1) Taking into account that knowledge distillation can “distillate” the feature information of network traffic that the large teacher model learns to the small student model. We transfer the representations of network traffic learned by the large model to the tiny model through knowledge distillation. (2) Considering that the depthwise separable convolution can extract network traffic features while using less computing power, we substitute depthwise separable convolution for the conventional convolution in order to significantly reduce the number of parameters as well as computational cost of the model. The lightweight intrusion detection models in previous papers mainly use the feature and algorithm perspective to lessen the model's complexity, such as using feature selection methods to choose the best feature subset, utilizing autoencoders to transfer features to a low-dimensional latent space, or applying machine learning algorithms rather than deep learning approaches to build intrusion detection models. Knowledge distillation has been described in Section 3.2, so we will explain in this section how to incorporate depthwise separable convolution into anomaly detection for IoT network.

The width, height, and channel dimension of each picture are among the three dimensions from which features are extracted by the typical convolution process. While depthwise separable convolution is performed by decoupling a complete convolution operation into two steps. First, all of the multi-channel feature maps from the preceding layer are divided into single-channel feature maps, and each of them is subjected to a separate single-channel convolution. The process of stacking them back together at the end is known as depthwise convolution. The

number of channels remains constant during the depthwise convolution process, it just resizes the feature map from the previous layer. Therefore, the feature map obtained by depthwise convolution will be convolved for the second time, and the convolution kernels in this convolution process are 1×1 in size, and the filter has the same number of convolution kernels as the preceding layer's output channels. For multiple channels, numerous filters are needed since a single filter produces a single feature map, which is called pointwise convolution. By decoupling a complete convolution into two steps, the depthwise separable convolution can avoid extracting some redundant features and greatly cut down on the quantity of model parameters thus reducing the risk of model overfitting issue. Fig. 5 illustrates the process of depthwise separable convolution.

Eq. (13) can be used to determine the number of convolution parameters for the depthwise convolution.

$$\text{num_params} = W_k * H_k * \text{in_channels} \quad (13)$$

The computational complexity of depthwise convolution can be obtained by following formula.

$$\text{Flops} = W_k * H_k * W_{\text{img}} * H_{\text{img}} * \text{in_channels} \quad (14)$$

where W_k and W_{img} are the width of the convolution kernel and the input feature map, H_k and H_{img} represent the height of the convolution kernel and the input feature map, respectively, and in_channels is the number of input channels.

For pointwise convolution, the number of convolution parameters can be expressed as follows.

$$\text{num_params} = 1 * 1 * \text{in_channels} * \text{out_channels} \quad (15)$$

The following formula can be used to determine the computational complexity of pointwise convolution.

$$\text{Flops} = 1 * 1 * W_{\text{feature}} * H_{\text{feature}} * \text{in_channels} * \text{out_channels} \quad (16)$$

where W_{feature} and H_{feature} are the width and height of the input feature map, and in_channels and out_channels indicate the number of input channels and output channels.

As for the conventional convolution, the number of convolution parameters is determined by Eq. (17).

$$\text{num_params} = W_k * H_k * \text{in_channels} * \text{out_channels} \quad (17)$$

The computational complexity of conventional convolution can be calculated by following formula.

$$\text{Flops} = W_k * H_k * W_{\text{feature}} * H_{\text{feature}} * \text{in_channels} * \text{out_channels} \quad (18)$$

After pre-processing, the network traffic is converted to $14 \times 14 \times 1$ grayscale map format, if the network traffic is not enough to convert into $14 \times 14 \times 1$, then the insufficient parts are used to fill with 0. In self-

knowledge distillation, the pre-processed network traffic is fed into the teacher model and student model after data augmentation, and the structure diagrams of the teacher model and student model are shown in Figs. 7 and 8. Therefore, we can calculate the number of parameters required for the teacher model and the student model as well as the computational complexity through Eqs. (13)–(18), where the number of parameters required for the teacher network and the student network (excluding the parameters in the classification header) are 346,912 and 13,401 respectively, while the computational complexity required for the teacher model and the student model (excluding the parameters in the classification header) are 15,893,632 and 673,828 respectively. Therefore, we can draw the conclusion that the number of parameters required for the student model is only 3.9 % of the teacher model, and the computational complexity required for the student model is only 4.2 % of the teacher model, thus we can save 96.1 % of the number of parameters and 95.8 % of the computational complexity by replacing the traditional convolution with depthwise separable convolution.

3.4. The steps of CL-SKD model for intrusion detection in IoT network

In this study, we employ the CL-SKD model to identify intrusions in IoT networks, while the datasets used are KDD CUP99, NSL-KDD, CIC IDS2017, UNSW-NB15 and BoT-IoT. The following are the precise stages of the CL-SKD model for intrusion detection.

Step 1: Using one-hot encoding and data normalization for the intrusion detection datasets.

Step 2: First stage of CL-SKD model:

- 1) Determine the Target network's and the Online network's structure, and initialize the parameters of Target network with the weights of Online network.
- 2) Input the augmented network traffic into Online network and Target network for training.
- 3) Using gradient descend algorithm to adjust the weight of the Online network until the Online network reaches convergence.
- 4) Save the weights of Online network locally to prepare for the second training stage.

Step 3: Second stage of CL-SKD model:

- 1) Determine the network structure of the student network and load the weights of Online network to the teacher network.
- 2) Input the augmented network traffic into teacher network and student network for training respectively.
- 3) Using gradient descend algorithm to adjust the weight of the student model until the student model reaches convergence.
- 4) Save the weights of student network locally to prepare for model testing.

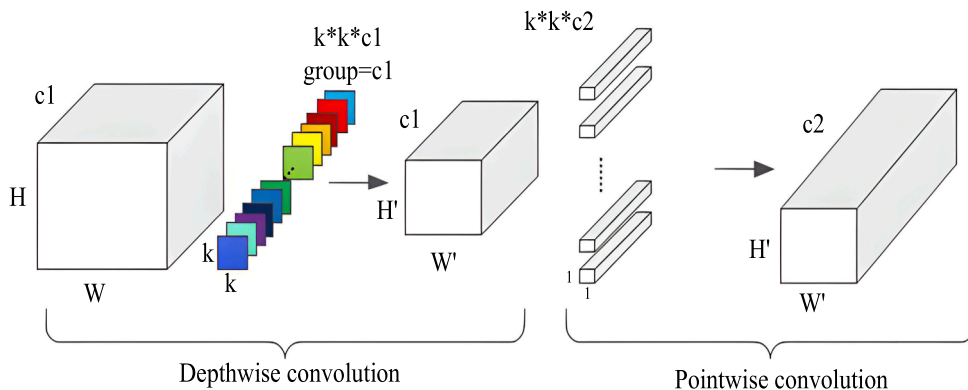


Fig. 5. The process of depthwise separable convolution.

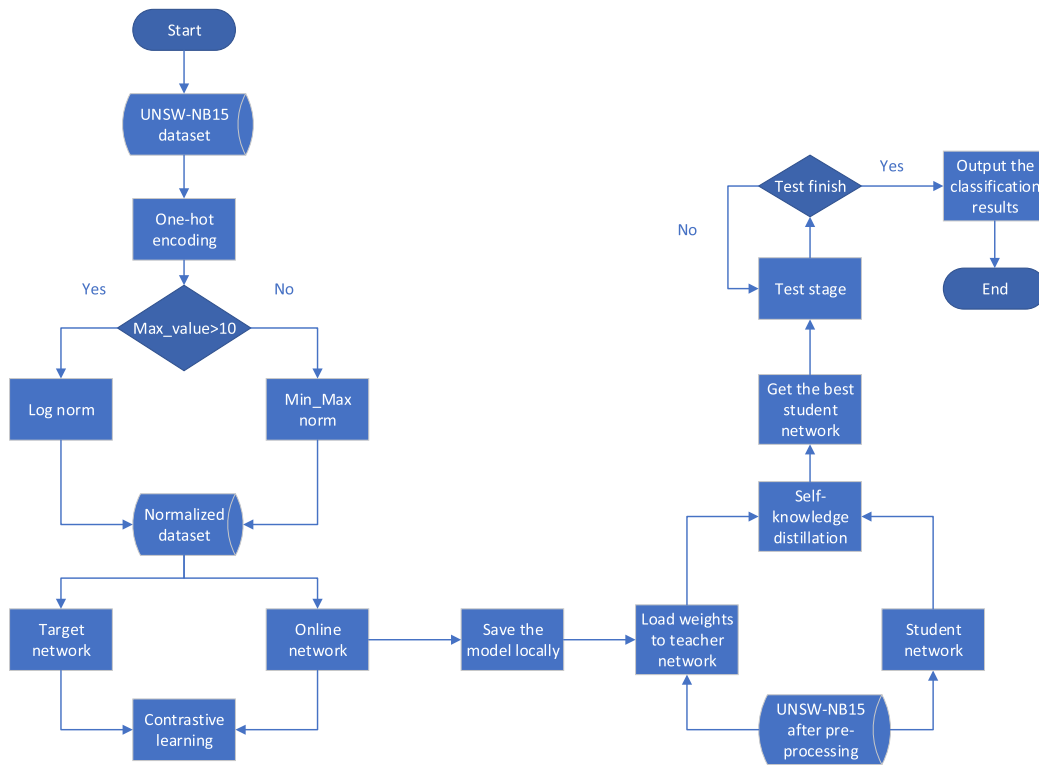


Fig. 6. Flow chart of the CL-SKD model.

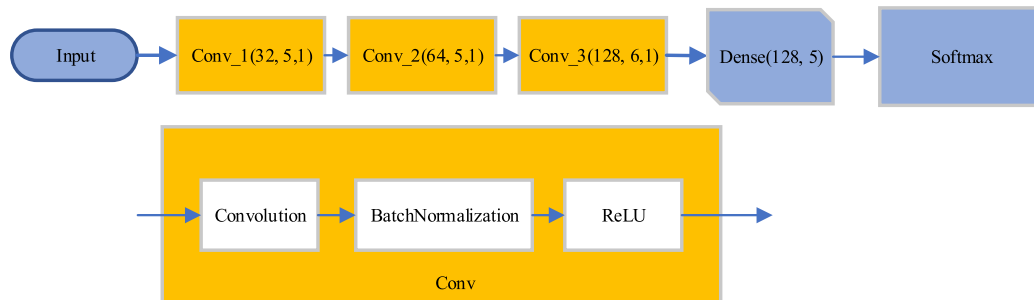


Fig. 7. Teacher model structure, 32/64/128 in Conv are the output channels, 5/5/6 in Conv are the kernel size, 1/1/1 in Conv are the stride, 128 in Dense is input dimension, 5 in Dense is the number of categories.

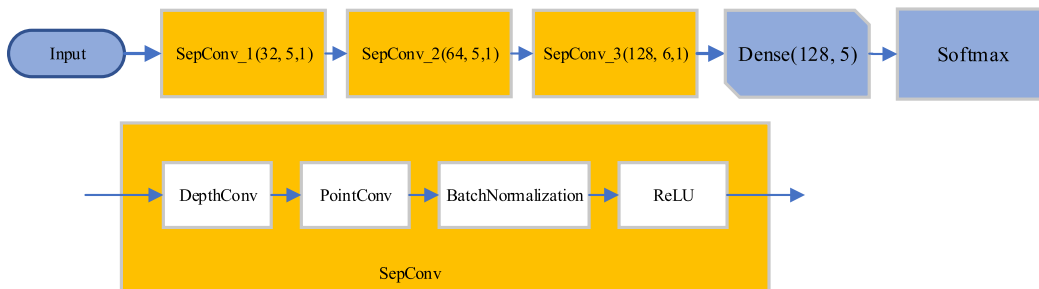


Fig. 8. Student model structure, 32/64/128 in SepConv are the output channels, 5/5/6 in SepConv are the kernel size, 1/1/1 in SepConv are the stride, 128 in Dense is input dimension, 5 in Dense is the number of categories.

Step 4: Testing for CL-SKD model. Load the weights of student model, then input the test dataset that has already been pre-processed into the student model to obtain the category for each network traffic.

Fig. 6 below displays the CL-SKD model's overall flow chart.

4. Datasets and pre-processing

Utilizing the IoT datasets KDD CUP99 (Tavallae et al., 2009), NSL-KDD (Dhanabal & Shantharajah, 2015), CIC IDS2017 (Sharafaldin, Lashkari, & Ghorbani, 2018), UNSW-NB15 (Moustafa & Slay, 2015) and

BoT-IoT (Koroniatis et al., 2019), we will conduct a large number of binary and multi classification experiments. Given that the UNSW-NB15 dataset has richer feature information, enough data, and a wider variety of assaults (Wang, Li, Wang, & Li, 2021), we apply self-supervised contrastive learning in UNSW-NB15 to get feature representations of network traffic and use self-knowledge distillation to transfer them to the student network. Next, we briefly describe the four benchmark datasets and explain how to pre-process the dataset.

4.1. Dataset description

The KDD CUP99 dataset has enormous quantity of normal and abnormal network traffic data, where the abnormal data can be subdivided into 4 major categories, named Dos, Probe, R2L and U2R. Each data has 41 features total—34 continuous features and 7 discontinuous features—as well as one label. Table 1 describes the distribution of the KDD CUP99 dataset.

The KDD CUP99 dataset has been optimized with the removal of redundant and duplicate data to create the NSL-KDD dataset. The NSL-KDD dataset is made up of the same five categories of data as the KDD CUP99: U2R, R2L, Dos, Normal and Probe. Compared with abnormal traffic, IoT devices are still accessed by normal traffic more often, so that we can analyse various anomalous traffic patterns to assess our model and simulate network traffic in actual IoT contexts using KDD CUP99 and NSL-KDD. Table 1 displays a comprehensive overview of the NSL-KDD dataset.

The Australian Centre for Cyber Security laboratory generated the UNSW-NB15 dataset, which is available to the public. Compared with the KDD CUP99 and NSL-KDD datasets, the UNSW-NB15 dataset contains a wider variety of attack methods. Each network traffic data consists of 47 features, 1 class label and 1 specific attack label. The UNSW-NB15 dataset contains novel, proportionate, and balanced anomalous traffic. It is hence appropriate as a dataset for IoT intrusion detection. The UNSW-NB15 dataset's data distribution is shown in Table 2.

The CIC IDS2017 dataset, which is representative of real-world data, includes recent benign and attack traffic, researchers collected a total of 2,830,743 IoT traffic samples for five days from Monday to Friday in a tiny simulated network environment. Limited by hardware limitations, we only use the IoT traffic collected on Wednesday as the intrusion detection dataset and split the dataset into training set and test set. Table 3 displays a comprehensive summary of the CIC IDS2017 dataset.

The BoT-IoT dataset includes a lot of DoS and DDoS attack along with benign and botnet network traffic that was recorded in a realistic network environment at UNSW Canberra. Limited by the computing capacity of the device as well as to save the training time, we choose UNSW_2018_IoT_Botnet_Final_10_best_Training.csv as the dataset, it is composed of 16 traffic features and 1 class label as well as 1 specific attack label. Table 4 details the data distribution of the BoT-IoT dataset.

4.2. Data pre-processing

Since the backbone of the self-supervised contrastive learning and self-knowledge distillation model we choose is convolutional neural network, the traffic data which is one dimensional data has to follow the

Table 1
Data distribution of KDD CUP99 and NSL-KDD.

Category	KDD CUP99		NSL-KDD	
	Train	Test	Train	Test
Normal	97,277	60,592	67,343	9711
Probe	4107	4166	11,656	2421
Dos	391,438	229,825	45,927	7460
U2R	52	228	52	67
R2L	1126	16,189	995	2885
Total	494,000	311,000	125,973	22,544

Table 2
Data distribution of UNSW-NB15.

Category	Train	Test
Normal	56,000	37,000
Fuzzers	18,184	6062
Analysis	2000	677
Backdoors	1746	583
Dos	12,264	4089
Exploits	33,393	11,132
Generic	40,000	18,871
Reconnaissance	10,491	3496
Shellcode	1133	378
Worms	130	44
Total	175,341	82,332

Table 3
Data distribution of CIC IDS2017.

Category	Train	Test
Benign	329,959	109,724
Dos Hulk	172,318	57,806
Dos Goldeneye	7739	2554
Dos Slowloris	4358	1438
Dos Slowhttptest	4170	1329
Heartbleed	10	1
Total	518,554	172,852

Table 4
Data distribution of BoT-IoT.

Category	Train	Test
Normal	284	86
DDoS	1,156,296	385,019
DoS	989,898	330,250
Reconnaissance	54,589	18,330
Theft	45	20
Total	2,201,112	733,705

convolutional neural network's input format, so we need to pre-process the intrusion detection datasets, which mainly includes three steps: one-hot encoding, data normalization and data augmentation.

1) One-hot encoding

Consider the UNSW-NB15 as an example, it contains the three non-numeric features of proto, state, and service, we must translate the non-numeric features into numerical features since the convolutional neural network requires numerical input data.

2) Data normalization

The most popular normalization technique in machine learning, Min_Max normalization, has several shortcomings, i.e., the normalization results may become unstable if the data's maximum and lowest values are considerably apart from one another. Therefore, we use a hybrid data normalization method to normalize the data as shown in Eq. (19).

$$\bar{x}_i = \begin{cases} \frac{x_i - x_{i_{\min}}}{x_{i_{\max}} - x_{i_{\min}}}, & x_{i_{\max}} < 10 \\ \log_{10}(1 + x_i), & x_{i_{\max}} \geq 10 \end{cases} \quad (19)$$

where $\bar{x}_i$ is the normalized result, x_i denotes the value of the i -th feature, $x_{i_{\min}}$ is the minimum value in the i -th feature, and $x_{i_{\max}}$ represents the maximum value in the i -th feature.

3) Data augmentation

After above two steps, the network traffic sample in UNSW-NB15 dataset consists of 196-dimensional vector. To follow the convolutional neural network's input format, we need to reshape the vector to tensor whose shape is $14 \times 14 \times 1$. Then, we use python library torchvision as computer vision domain does to perform data augmentation on the tensor data. We choose RandomCrop, RandomHorizontalFlip and RandomVerticalFlip operations to augment the network traffic data. By randomly cropping the network traffic data, we introduce variability, which helps the model generalize better. This is akin to performing random selection in the feature space, thereby increasing the diversity of the training data and reducing the risk of overfitting. Random flipping similarly increases data diversity. While this operation has spatial significance in image processing, in our network traffic data, it simulates a reordering of features, making the model more robust.

5. Experimental setup

We will utilize accuracy, precision, recall, F1-measure, and ROC curve as assessment measures to accurately evaluate the efficiency and performance of the proposed CL-SKD model with existing models, and the formulas for these indicators can be found in (Wang, Li, He, & Chan, 2022).

5.1. Benchmark models

To comprehensively evaluate the proposed model, we use a range of state-of-the-art models as baselines to compare with our model. Which are listed below:

CCFS-DT (Farahani, 2020) used cross-correlation-based feature selection to obtain the best feature subset, and trained decision tree with such feature subset. The maximum depth of the DT model is 3, and its feature selection criterion is the Gini coefficient.

GTO-BSA (Kareem et al., 2022) is a new feature selection algorithm based on Gorilla Troops Optimizer and bird swarms, which can be used for choosing the optimal feature subset of intrusion detection datasets. The paper employs KNN as the classifier, with the number of neighbors for classification set to 5 and Euclidean distance used as the distance metric.

NB-SVM (Gu & Lu, 2021) utilized naïve Bayes to obtain new data with high quality, an SVM classifier is used for intrusion detection based on the new data. The SVM model uses radial basis function as the kernel function, and the penalty coefficient of the objective function is 1.0.

TODE (Fatani et al., 2021) is proposed to identify intrusions in IoT traffic which used improved TSO algorithm to select best feature subset, and then input into machine learning classifier to identify intrusions. The paper also employs KNN as the classifier, with the number of neighbors for classification set to 5 and Euclidean distance used as the distance metric.

ET, SVM and DT are machine learning methods from the paper (Sharma & Yadav, 2021), the authors used Recursive Feature Elimination to remove irrelevant features from original dataset and applied these machine learning methods to classify attacks. The maximum depth of the ET and DT model is 4, and its feature selection criterion is the Gini coefficient, while the SVM model uses radial basis function as the kernel function, and the penalty coefficient of the objective function is 1.0.

IBYOL-IDS (Wang, Li, Wang, & Li, 2021) is an intrusion detection model based on BYOL self-supervised learning, which is pre-trained on UNSW-NB15 without labeled data.

CMAE (Lu et al., 2021) is proposed to solve the data imbalance issue in field of intrusion detection by leveraging a memory module to store normal traffic representations while also benefiting from the strengths of an autoencoder.

In order to address the issue of long training time and poor performance of existing models, SDAE-ELM (Wang, Liu, He, & Chan, 2021) used small batch gradient descent method and extreme learning machine to overcome above shortcomings.

SS-Deep-ID (Abdel-Basset et al., 2021) is a method for classifying attacks in IoT traffic based on semi-supervised learning, and the method can be integrated into a fog-enabled IoT network.

E-GraphSAGE (Lo et al., 2022) viewed network traffic as graph-structured data, and apply GNN for intrusion detection.

LSTM-FCNN (Sahu et al., 2022) combined LSTM with FCN to accurately classify the normal and malicious connections on intrusion datasets.

KD-TCNN (Wang, Li, He, & Chan, 2022) is proposed to reduce the complexity of the intrusion detection model as well as improve the model performance, which was based on deep metric learning and knowledge distillation.

Data preprocessing which may cause loss in intrusion detection was discarded in LOGNN (Wang, Xu, He, & Chan, 2021), the authors designed a new type neural network based on logarithm neuron and applied it to detect abnormal traffic.

Above methods can be classified into machine learning based models and deep learning based models. NB-SVM, TODE, ET, SVM, DTCCFS-DT and GTO-BSA are machine learning based intrusion detection methods, while IBYOL-IDS, CMAE, SDAE-ELM, SS-Deep-ID, E-GraphSAGE, LSTM-FCNN, KD-TCNN and LOGNN are deep learning based intrusion detection methods and Table 5 presents the implementation parameters of the SOTA models and CL-SKD model.

5.2. Implementation details of the CL-SKD model

The experimental results are calculated by averaging the results obtained from 3 different sets of random seeds. In the first training stage, the batch size is 1024, and the learning rate is $1e-3$, we use the ReLU activation function, AdamW as the gradient optimizer, the momentum coefficient η is 0.999, the memory queue length is 10240, the K is 5, and we train 20 epochs. In the second training stage, our learning rate, batch size, activation function, gradient optimizer and training epochs are the same as in the first training stage, the memory queue length is set to 65536, the temperature parameters of the student and teacher models are 0.07 and $1e-4$, respectively, and k is set to 100. For intrusion detection in IoT, limited by the computational resources and storage capacity of IoT devices, we need to make a trade-off between detection efficiency and model size. Therefore, we prefer to use a CNN with only three convolutional layers as the intrusion detection model (Fewer layers means fewer parameters and calculations), just as same architecture as LuNet (Wu & Guo, 2019). Figs. 7 and 8 display the topologies of the teacher model and the student model in finetuning stage and linear evaluation stage, respectively.

6. Experimental results and discussion

6.1. Analysis of the effectiveness of the improvement methods

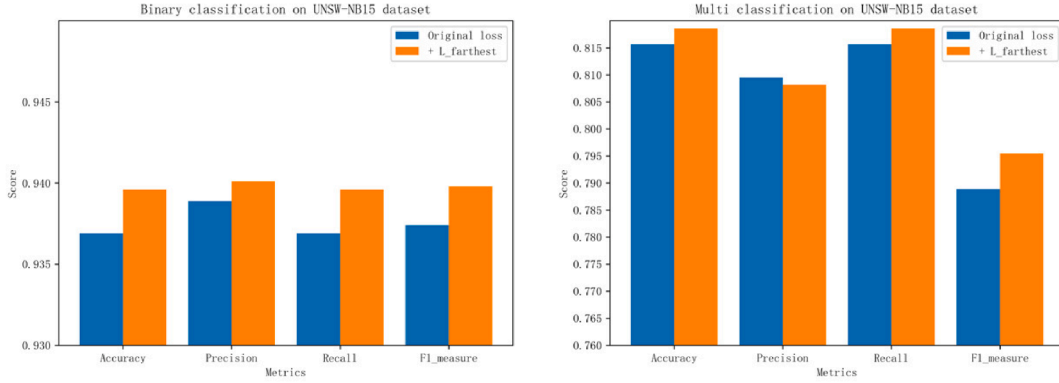
In Section 3.1, we proposed to find the set of feature embeddings $\{f_i\}_{i=1}^K$ obtained from the K farthest neighbours of $\bar{z}_2$ at the early training phase in self-supervised contrastive learning, and use the average distance $L_{farthest}$ between this set of feature embeddings $\{f_i\}_{i=1}^K$ and $\bar{q}$ as the component of the cost function. To confirm the effectiveness of this improvement, we include whether to add $L_{farthest}$ as a variable and keep the other parameters unchanged, and then the binary and multi classification results obtained by fine-tuning on the UNSW-NB15 dataset are used as the judgment criteria, and Fig. 9 displays the experimental results. As seen in Fig. 9, after adding $L_{farthest}$ as a part of the loss function, the model has better intrusion detection effect on the UNSW-NB15 dataset, which fully verifies the effectiveness of our proposed improvement method. As a result, we use the improvement version of self-supervised contrastive learning to train the CL-SKD model in subsequent experiments.

In Section 3.2, we propose to find the k nearest neighbours of feature

Table 5

Implementation parameters of the SOTA deep learning models and CL-SKD model.

Model	Lr	epoch	optimizer	batch_size	active_function	parameters
IBYOL-IDS	1e-4	20	Adam	512	ReLU	1,578,145
CMAE	1e-3	100	Adam	128	ReLU	130,577
SDAE-ELM	1e-4	100	Adam	100	ReLU	44,293
SS-Deep-ID	3e-4	20	Adam	256	ReLU	663,434
E-GraphSAGE	1e-3	700	Adam	512	ReLU	94,722
LSTM-FCNN	1e-4	10	Adam	10	ReLU	1,137,157
KD-TCNN	1e-4	40	Adam	512	Swish	4624
LOGNN	1e-4	20	Adam	512	Tanh	12,333
CL-SKD	1e-3	20	AdamW	1024	ReLU	14,494

**Fig. 9.** Comparison results of Fine-tuning experiments on UNSW-NB15 dataset with improved self-supervised learning.

embedding $\bar{y}_i^T$ in the memory queue Q , and then compute the distances between them, which can be denoted as $P_T(x_i, \theta, Q^T)$. Similarly, we can find the k nearest neighbours of feature embedding $\bar{y}_i^S$ in the memory queue Q , and then compute the distances between them, which can be denoted as $P_S(x_i, \theta, Q^S)$. Then we calculate the cross-entropy loss of $P_T(x_i, \theta, Q^T)$ and $P_S(x_i, \theta, Q^S)$ as the loss function for self-knowledge distillation. To verify the validity of this improvement, we take whether to take k nearest neighbours as a variable and keep other parameters unchanged, and then perform self-knowledge distillation training on the UNSW-NB15, and use the average training time per epoch and the binary and multi classification results obtained by fine-tuning on the UNSW-NB15 dataset as the judgment criteria, and the experimental results are shown in Table 6. Where *Ori* represents the result obtained by using the original loss function, *Mod* represents the result obtained by using the improved loss function, *binary* represents the binary classification task, and *multi* represents the multi classification task, and training time is the time used to train self-knowledge distillation model, the unit of training time is seconds. As seen in Table 6, the results of the optimized loss function can exceed the results of the original loss function for both binary and multi classification task, and the training time spent also decreases, which confirms in full the efficacy of our suggested enhanced method, therefore, in the subsequent experiments we use the optimized loss function to train the CL-SKD model.

Table 6

Comparison results of improved self-knowledge distillation on the UNSW-NB15 dataset for Fine-tuning experiments.

Model	Accuracy	Precision	Recall	F1-measure	Training time
Ori_binary	93.78	93.90	93.78	93.81	31.32
Mod_binary	93.90	93.92	93.90	93.91	30.76
Ori_multi	80.96	79.88	80.96	78.57	31.32
Mod_multi	81.24	80.19	81.24	78.56	30.76

6.2. Analysis of the lightweight model

To validate the lightness of the CL-SKD model, we employ traditional deep learning models, such as DNN, CNN, and RNN to carry out multi-classification tasks on the NSL-KDD dataset. Where the architectures of the former two are kept identical to the student model architecture, and the latter consists of a layer of output layers and a LSTM layer of 128 neurons. We use not only accuracy and F1-measure as metrics, but also the number of model parameters, model size as well as model computation Flops as indicators of model lightweighting. Table 7 displays the experimental results in detail, where the value inside the parentheses is the performance difference from the CL-SKD model.

As seen in Table 7 the CL-SKD model achieves better performance both in classification accuracy and model lightweight metrics. In the case of similar accuracy, the CNN model has 24 times higher parameter count and 16 times higher computation cost than the CL-SKD model, while in the case of similar model size, the DNN model is less than the CL-SKD model in terms of accuracy by 0.8 %, and the F1-measure is lower than 0.81 %. For the RNN model, a lower accuracy and F1-measure are achieved with relatively larger number of model parameters, which is due to the RNN model's over-reliance on input data that leads to model overfitting. Overall, due to the simplicity of the NSL-KDD dataset and the small variety of attack categories, all deep learning methods can produce reasonably classification results on the NSL-KDD

Table 7

Classification results of different deep learning models in NSL-KDD.

Model	Accuracy	F1-measure	Params	Size	Flops
DNN	99.15(↓0.80)	99.14(↓0.81)	14,981	58.52 KB	15,106,048
CNN	99.42(↓0.53)	99.41(↓0.54)	348,229	1.33 MB	2,295,070,720
RNN	98.66(↓1.29)	98.65(↓1.30)	129,157	504.52 KB	133,300,224
CL-SKD	99.95	99.95	14,494	56.62 KB	145,297,408

dataset.

6.3. Binary classification

We classify all attack traffic as abnormal category, and intrusion detection becomes a binary classification problem. We compare the CL-SKD model with latest state-of-the-art models using KDD CUP99, NSL-KDD, CIC IDS2017, and BoT-IoT datasets to confirm the superiority of the model provided in our study. Tables 8 through 11 display the experimental findings. The notation “-” indicates that the metric’s result was not provided in the original study, CL-SKD-L is the result obtained from CL-SKD model performing Linear evaluation (freezing the feature extraction layer and training the classification head only) on the above dataset, CL-SKD-F is the result obtained from CL-SKD model performing fine-tuning (fine tune the feature extraction layer and training the classification head) on the above dataset.

Table 8 illustrates that all models have an accuracy of greater than 93 %, this is mostly due to the KDD CUP99 dataset’s simplicity and abundance of duplicate information, all the models can effectively extract the features of traffic data to obtain excellent classification results. In terms of precision, the CL-SKD-F model can obtain the best accuracy of 99.95 %, which is 3.28 % higher than the CMAE model. In the aspect of recall and F1-measure, the CL-SKD model can also acquire the best results. As can be seen from Table 9, because the KDD CUP99 dataset’s data redundancy issue is resolved by the NSL-KDD dataset, the CL-SKD-L and the CL-SKD-F model slightly decrease in accuracy, while the CL-SKD-F model is still able to surpass the IBYOL-IDS, NB-SVM and SADE-ELM by 6.76 %, 0.08 % and 21.39 %. The CL-SKD-F model can even transcend the SADE-ELM model by 35.31 % in recall, which is mainly because the CL-SKD model is able to learn the features that distinguish the essence of abnormal traffic data to achieve such excellent detection ability. In terms of F1-measure, the CL-SKD-F model is 2.52 % higher than the CL-SKD-L model, which indicates that slightly modifying the weights of the feature extraction layer can achieve better detection efficiency on the NSL-KDD dataset. From Tables 10 and 11, it can be seen that the CL-SKD models are able to achieve the best intrusion detection results on the CIC IDS2017 and BoT-IoT datasets, and due to the sufficient amount of data in the above two datasets, each model is able to fully learn to discriminate the abnormal traffic representations. Hence, all models are able to achieve more than 97 % of the binary classification accuracy and F1-measure on above two datasets. In particular, the CL-SKD-F model is able to achieve more than 99.80 % for each metric on both datasets. On the CIC IDS2017 dataset, both the CL-SKD models are able to achieve comparable or better detection results than IBYOL-IDS, NB-SVM and SS-Deep-ID, while on the BoT-IoT dataset, the CL-SKD-F model is able to outperform TSOE and E-GraphSAGE in terms of accuracy by 0.01 %, respectively. What’s more, the CL-SKD-F model and CL-SKD-L model as well as ET model get perfect binary classification results in BoT-IoT.

The fact that the ROC curve’s form may essentially stay the same even when the distribution of positive and negative samples shifts is a major benefit over the P-R curve. The CL-SKD-L and CL-SKD-F ROC curves for the KDD CUP99, NSL-KDD, CIC IDS2017, and BoT-IoT datasets are displayed in Fig. 10, respectively. From Fig. 10 upper left and Fig. 10 lower right, it is evident that the CL-SKD-F can achieve the AUC value of 1.00 on the KDD CUP99 and BoT-IoT datasets. As Fig. 10 upper

Table 8
Experimental results of binary classification of each model on KDD CUP99.

Model	Accuracy	Precision	Recall	F1-measure
IBYOL-IDS	99.25(↓ 0.7)	99.74(↓ 0.21)	99.33(↓ 0.62)	99.53(↓ 0.42)
CMAE	—	96.67(↓ 3.28)	98.30(↓ 1.65)	97.48(↓ 2.47)
SADE-ELM	93.57(↓ 6.38)	98.69(↓ 1.26)	93.19(↓ 6.76)	95.86(↓ 4.09)
CL-SKD-L	99.90(↓ 0.05)	99.90(↓ 0.05)	99.90(↓ 0.05)	99.90(↓ 0.05)
CL-SKD-F	99.95	99.95	99.95	99.95

Table 9

Experimental results of binary classification of each model on NSL-KDD.

Model	Accuracy	Precision	Recall	F1-measure
IBYOL-IDS	92.67(↓ 6.76)	92.48(↓ 6.95)	92.06(↓ 7.37)	92.27(↓ 7.16)
NB-SVM	99.35(↓ 0.08)	—	99.24(↓ 0.19)	—
SADE-ELM	78.04(↓ 21.39)	95.99(↓ 3.44)	64.12(↓ 35.31)	76.87(↓ 22.56)
CL-SKD-L	96.91(↓ 2.52)	96.91(↓ 2.52)	96.91(↓ 2.52)	96.91(↓ 2.52)
CL-SKD-F	99.43	99.43	99.43	99.43

Table 10

Experimental results of binary classification of each model on CIC IDS2017.

Model	Accuracy	Precision	Recall	F1-measure
IBYOL-IDS	96.70(↓ 3.1)	95.24(↓ 4.56)	95.76(↓ 4.04)	95.50(↓ 4.30)
NB-SVM	99.35(↓ 0.45)	—	99.24(↓ 0.56)	—
SS-Deep-ID	99.33(↓ 0.47)	99.47(↓ 0.34)	99.23(↓ 0.57)	99.35(↓ 0.45)
CL-SKD-L	99.33(↓ 0.47)	97.38(↓ 2.43)	99.33(↓ 0.47)	97.34(↓ 2.46)
CL-SKD-F	99.80	99.81	99.80	99.80

Table 11

Experimental results of binary classification of each model on BoT-IoT.

Model	Accuracy	Precision	Recall	F1-measure
TSODE	99.99(↓ 0.01)	99.99(↓ 0.01)	99.99(↓ 0.01)	99.99(↓ 0.01)
E-GraphSAGE	99.99(↓ 0.01)	100.00	99.99(↓ 0.01)	100.00
ET	100.00	—	100.00	100.00
CL-SKD-L	100.00	100.00	100.00	100.00
CL-SKD-F	100.00	100.00	100.00	100.00

right illustrates that with the reduction of data volume and the elimination of data redundancy issue, the AUC values of the CL-SKD-L and the CL-SKD-F on the NSL-KDD dataset slightly decrease, but the CL-SKD-F model is still 0.02 higher than the CL-SKD-L model. From Fig. 10 lower left, we may thus infer that in the CIC IDS2017 dataset, the highest difference in AUC value between the two models is 0.03, and the CL-SKD-F model is still able to reach an AUC value of 1.00.

In summary, taking into account that the CL-SKD model can pick up the fundamental feature representations of both normal and abnormal traffic data, both the CL-SKD-L model and the CL-SKD-F model can achieve excellent detection results and outperform the recent state-of-the-art models in the above datasets. We can also find that the CL-SKD model has significant generalization capacity to learn the traffic data representations as well as improve the intrusion detection performance without changing the weight of the feature extraction layer and only training the classification head.

6.4. Multi classification

We divide abnormal traffic into specific attack categories, and intrusion detection becomes a multi classification problem. To confirm that the proposed model in our study has excellent network traffic discrimination capability, we also conduct extensive multi classification experiments on the KDD CUP99, NSL-KDD, CIC IDS2017 and BoT-IoT datasets. As the same in the binary classification experiments, we compare the CL-SKD model with the recent state-of-the-art methods on above datasets. Tables 12 through 15 display the experimental findings.

Table 12 shows that even if the task’s complexity has increased, i.e., from a binary classification task to a multi classification task, the accuracy difference of the CL-SKD-L model between binary and multi classification is only 0.02 % on the KDD CUP99 dataset. As far as the CL-SKD-F model, its multi classification accuracy is even 0.01 % higher than the binary classification. In contrast to the other models, the CL-SKD-F model is higher than SVM, DT and LSTM-FCNN by 1.21 %, 4.71 % and 1.06 % in terms of accuracy, respectively. The CL-SKD-F model is greater than SVM, DT and LSTM-FCNN by 1.21 %, 4.21 % and 1.16 % in the

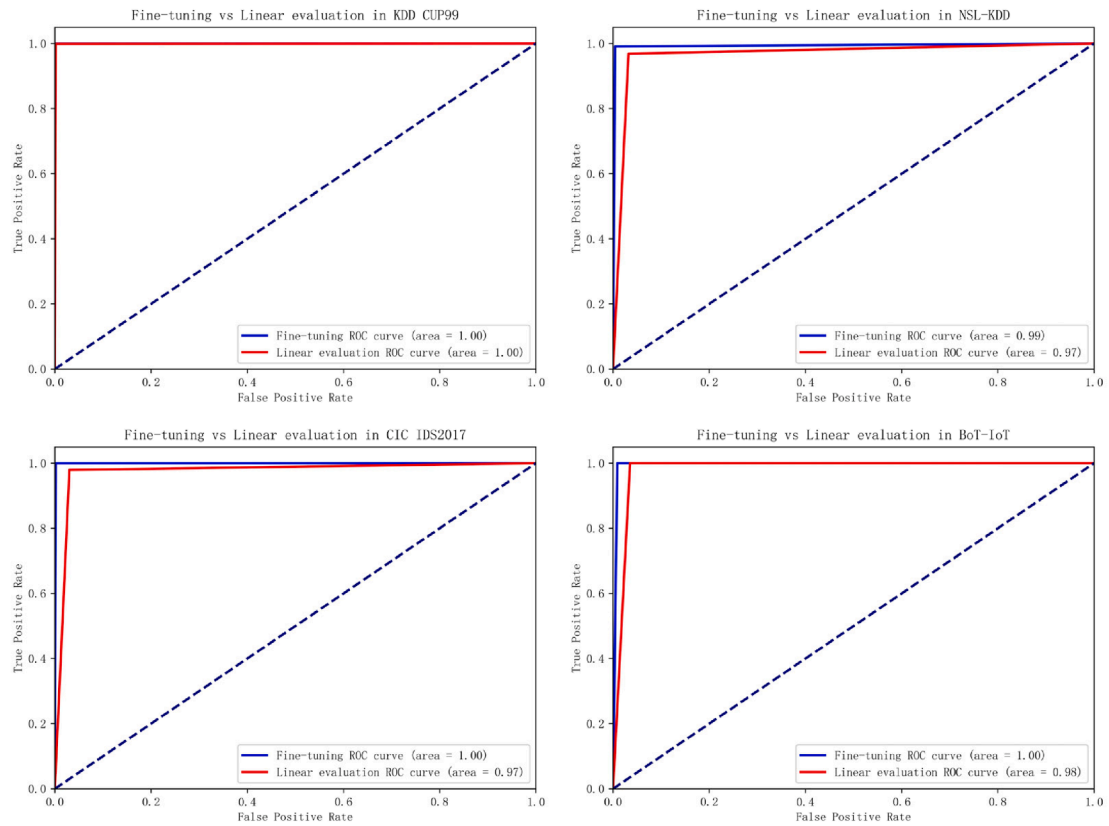


Fig. 10. ROC curve of CL-SKD-L and CL-SKD-F model on intrusion detection datasets.

Table 12

Experimental results of multi classification of each model on KDD CUP99.

Model	Accuracy	Precision	Recall	F1-measure
SVM	98.00(↓ 1.96)	98.75(↓ 1.21)	97.75(↓ 2.21)	98.75(↓ 1.21)
DT	98.50(↓ 1.46)	95.25(↓ 4.71)	96.25(↓ 3.71)	95.75(↓ 4.21)
LSTM-FCNN	98.52(↓ 1.44)	98.90(↓ 1.06)	98.70(↓ 1.26)	98.80(↓ 1.16)
CL-SKD-L	99.92(↓ 0.04)	99.91(↓ 0.05)	99.92(↓ 0.04)	99.91(↓ 0.05)
CL-SKD-F	99.96	99.96	99.96	99.96

aspect of F1-measure, respectively. From the perspective of the overall metrics, the difference between the results of the CL-SKD-L model and the CL-SKD-F model is very small. As is shown in Table 13, due to the reduction of training data volume and the increase of task complexity, the detection results of the CL-SKD-L model and the CL-SKD-F model in NSL-KDD dataset are somewhat different. The CL-SKD-L model is lower than the CL-SKD-F model by 2.4 % under the overall metrics. Compared to the other models, the CL-SKD-F model is higher than the KD-TCNN model by 1.35 % in precision. In terms of F1-measure, the CL-SKD-F model is greater than the LSTM-FCNN, KD-TCNN and LOGNN by 0.95 %, 1.44 % and 1.45 %, respectively. As can be seen from Table 14, the difference between the detection results of the CL-SKD-L model and the CL-SKD-F model in the CIC IDS2017 dataset further decreases since there are more training data available, and the CL-SKD-L model is worse

Table 13

Experimental results of multi classification of each model on NSL-KDD.

Model	Accuracy	Precision	Recall	F1-measure
LSTM-FCNN	98.94(↓ 1.01)	98.90(↓ 1.05)	99.10(↓ 0.85)	99.00(↓ 0.95)
KD-TCNN	98.44(↓ 1.51)	98.60(↓ 1.35)	98.47(↓ 1.48)	98.51(↓ 1.44)
LOGNN	98.70(↓ 1.25)	98.40(↓ 1.55)	98.70(↓ 1.25)	98.50(↓ 1.45)
CL-SKD-L	97.55(↓ 2.40)	97.50(↓ 2.45)	97.55(↓ 2.40)	97.49(↓ 2.46)
CL-SKD-F	99.95	99.95	99.95	99.95

Table 14

Experimental results of multi classification of each model on CIC IDS2017.

Model	Accuracy	Precision	Recall	F1-measure
KD-TCNN	99.44(↓ 0.40)	99.48(↓ 0.36)	99.47(↓ 0.37)	99.46(↓ 0.38)
LSTM-FCNN	99.49(↓ 0.35)	100(↑ 0.16)	99.60(↓ 0.24)	99.70(↓ 0.14)
CCFS-DT	97.91(↓ 1.93)	99.42(↓ 0.42)	99.07(↓ 0.77)	99.24(↓ 0.60)
CL-SKD-L	97.88(↓ 1.96)	97.91(↓ 1.93)	97.88(↓ 1.96)	97.88(↓ 1.96)
CL-SKD-F	99.84	99.84	99.84	99.84

than the CL-SKD-F model by 1.96 % under the overall metrics. Particularly, the LSTM-FCNN model is able to achieve 100 % precision in the CIC IDS2017 dataset, which is even 0.16 % higher than the CL-SKD-F model. In terms of recall, the CL-SKD-F model is 0.40 %, 0.24 %, and 0.77 % higher than the KD-TCNN, LSTM-FCNN, and CCFS-DT, respectively. As far as F1-measure, the CL-SKD-F model outperforms the KD-TCNN, LSTM-FCNN and CCFS-DT by 0.38 %, 0.14 % and 0.6 %, which shows that the CL-SKD-F model has similar intrusion detection results to the KD-TCNN, LSTM-FCNN and CCFS-DT in the CIC IDS2017 dataset. Table 15 shows that, on the BoT-IoT dataset, the accuracy of the TSDODE model is 1.5 % better than that of the CL-SKD-F model, but 0.93 % worse on other measures. The CL-SKD-F model is 5.12 % higher than the GTO-BSA in terms of accuracy. In the aspect of F1-measure, the ET model is able to achieve the best F1-measure, which is 100.00 %, because ET

Table 15

Experimental results of multi classification of each model on BoT-IoT.

Model	Accuracy	Precision	Recall	F1-measure
TSDODE	99.04(↓ 0.93)	99.04(↓ 0.93)	99.04(↓ 0.93)	99.04(↓ 0.93)
GTO-BSA	94.85(↓ 4.12)	—	99.28(↓ 0.69)	—
ET	—	—	100.00(↑ 0.03)	100.00(↑ 0.03)
CL-SKD-L	97.54(↓ 2.43)	97.55(↓ 2.42)	97.54(↓ 2.43)	97.54(↓ 2.43)
CL-SKD-F	99.97	99.97	99.97	99.97

model uses supervised learning. Overall, in the aforementioned datasets, every model is able to obtain acceptable multi classification performance metrics, demonstrating its ability to identify and categorize IoT network traffic, but the testing results of CL-SKD-L and CL-SKD-F are noticeably superior to those of the state-of-the-art models.

Confusion matrix shows the difference between different categories of data by using heat map, colour difference and brightness, it is a standard format to represent accuracy evaluation and also a tool to calculate each evaluation metric, it can visualize the records in the dataset by aggregating the actual and prediction results, thus helping us to judge the goodness of the model. Figs. 11 and 12 show the confusion matrix of the CL-SKD-L and CL-SKD-F on the KDD CUP99, NSL-KDD, CIC IDS2017 and BoT-IoT datasets, respectively. Comparing Figs. 11(a) and 12(a), the CL-SKD-L model can only identify 44 % of the U2R and classify 25 % of the U2R as Normal, while the CL-SKD-F model can identify 88 % of the U2R. Comparing Fig. 11(b) and 12(b), the CL-SKD-L model can only identify 19 % of U2R and classify 63 % of U2R as Normal, while the CL-SKD-F model can identify 74 % of U2R, which indicates that the CL-SKD-F model performs better than the CL-SKD-L model in the case of insufficient samples, which also reflects the importance of data volume on deep learning from the side. From Fig. 11

(d) and 12(c), (d), on the CIC IDS2017 and BoT-IoT datasets, it is evident that both the CL-SKD-L and CL-SKD-F models are able to obtain outstanding multi classification results with an increase in data volume, and the accuracy of each category can exceed 80 % or more, and even can basically reach 100 % detection rate in some categories. It further establishes beyond doubt the efficacy of the CL-SKD model put out in this work. From Fig. 11(c), it is surprising for the CL-SKD-L model to success to identify the Heartbleed category well due to the extremely sparse amount of Heartbleed category in training data.

6.5. Hyperparameter sensitive analysis

In order to verify the impact of key hyperparameters (including memory bank size and K value in K-nearest neighbour in self-supervised contrastive learning, and memory bank size and k value in K-nearest neighbour in self-knowledge distillation) on intrusion detection performance in this paper, we perform self-supervised contrastive learning and self-knowledge distillation under different hyperparameter settings on the UNSW-NB15 dataset and multi-classification fine-tuning under the UNSW-NB15 dataset, and the experimental results are shown in Figs. 13 and 14.

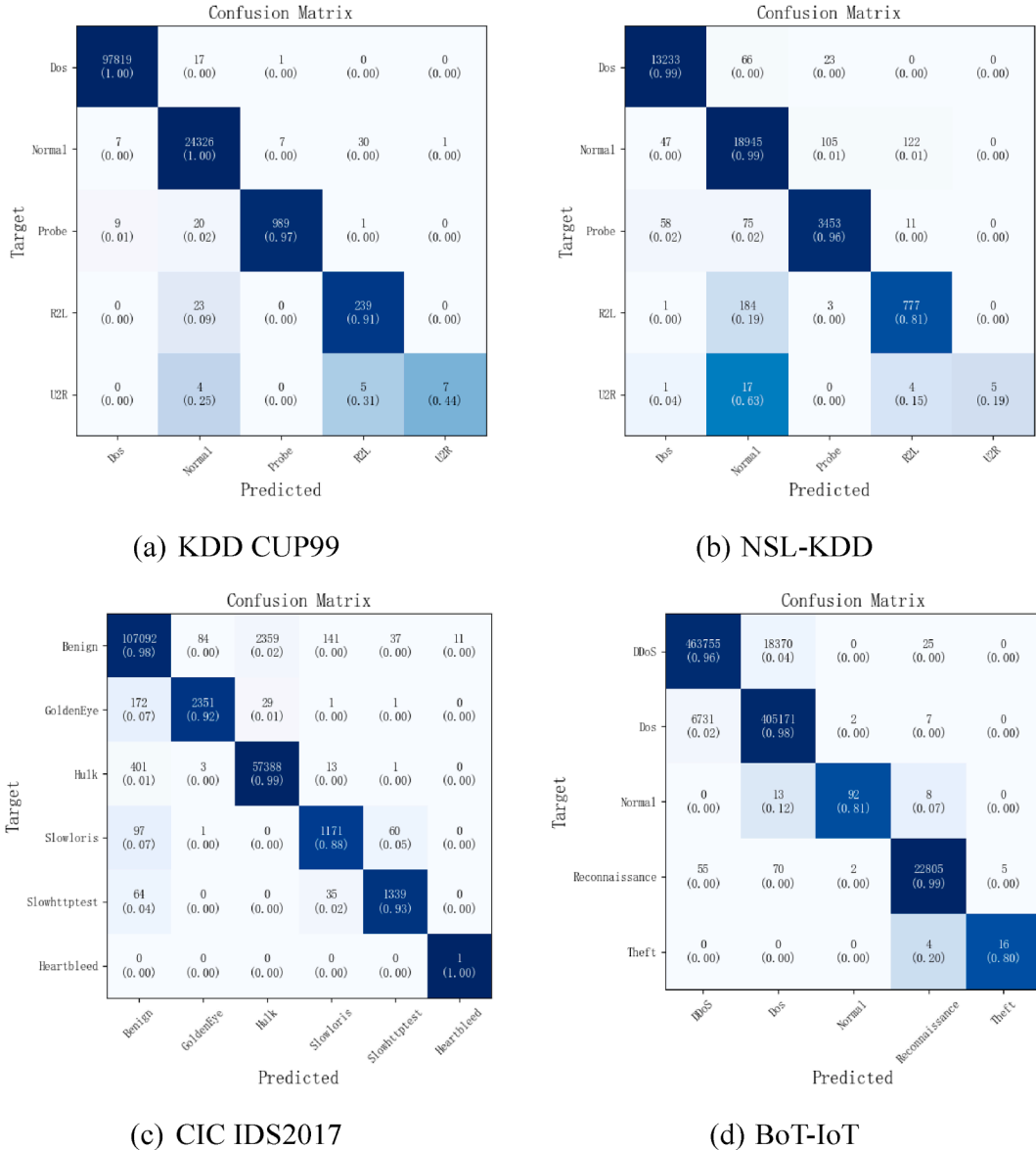
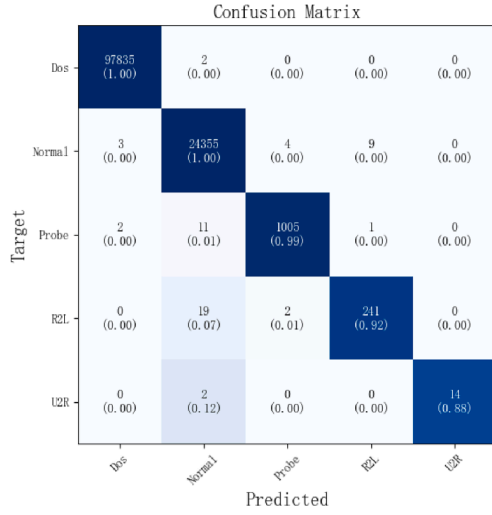
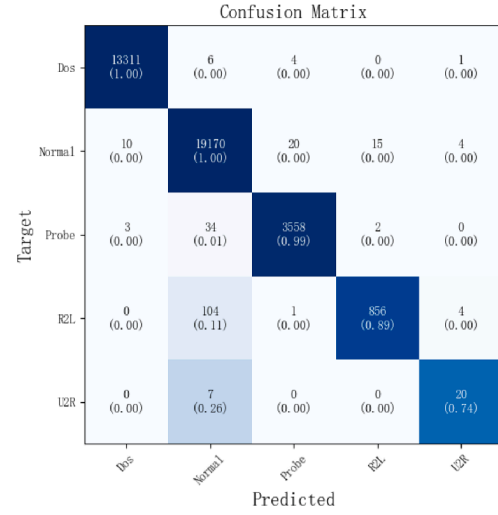


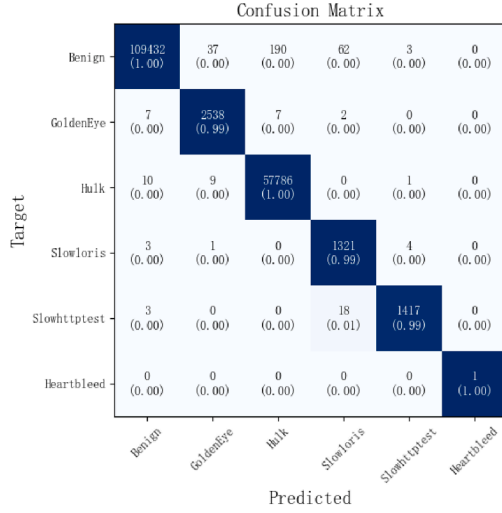
Fig. 11. Confusion matrix of CL-SKD-L model on intrusion detection datasets.



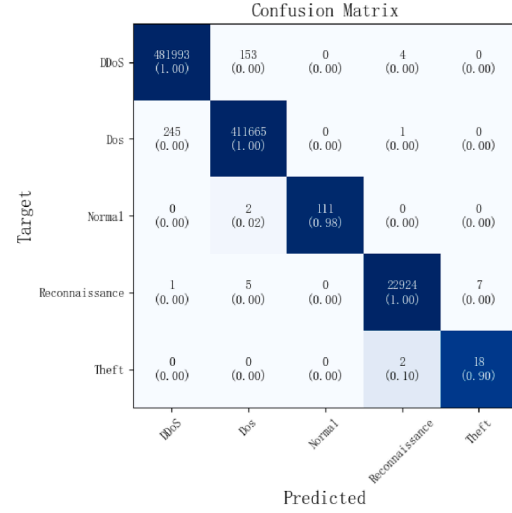
(a) KDD CUP99



(b) NSL-KDD

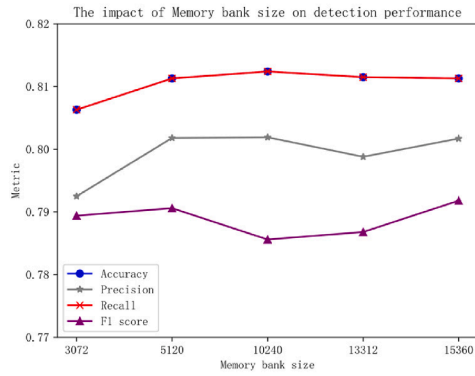


(c) CIC IDS2017

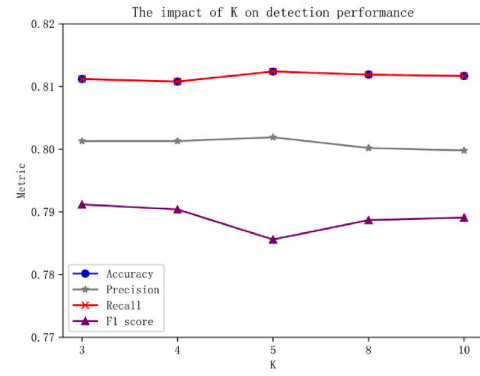


(d) BoT-IoT

Fig. 12. Confusion matrix of CL-SKD-F model on intrusion detection datasets.



(a) The impact of bank size in SSL stage



(b) The impact of K in SSL stage

Fig. 13. The impact of different hyperparameters in self-supervised learning stage.

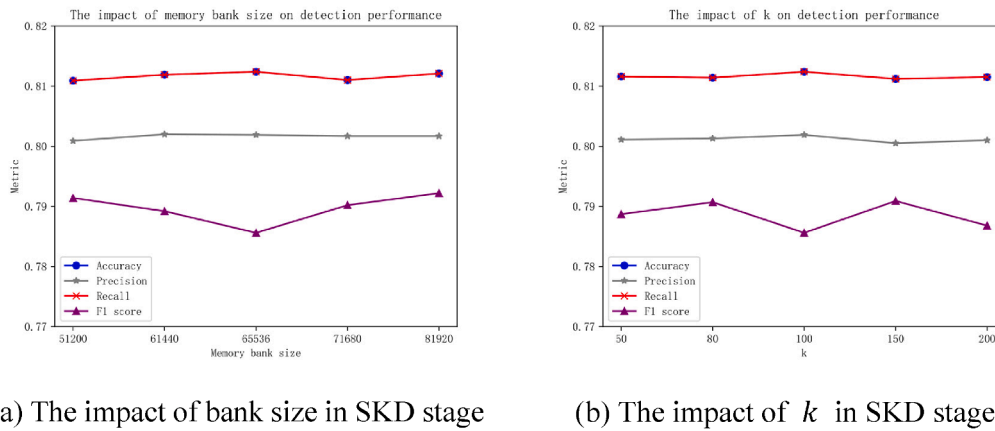


Fig. 14. The impact of different hyperparameters in self-knowledge distillation stage.

According to Fig. 13, it can be observed that during the self-supervised learning phase, when the memory bank size is set to 10,240 and K is set to 5, the UNWS-NB15 multi-class intrusion detection achieves the highest accuracy, precision, and recall. Moreover, the choice of memory bank size has a limited impact on the intrusion detection performance of the CL-SKD model. Furthermore, the CL-SKD model demonstrates good robustness to the hyperparameter K . According to Fig. 14, during the self-knowledge distillation phase, when the memory bank size is set to 65,536 and k is set to 100, the UNWS-NB15 multi-class intrusion detection achieves the highest accuracy, precision, and recall. Additionally, the CL-SKD model exhibits good robustness to the hyperparameters of memory bank size and k .

7. Conclusion and future work

In this research, we apply the concepts of self-supervised learning and self-knowledge distillation to present a two-stage lightweight intrusion detection model. In the first stage, we take advantage of improved self-supervised contrastive learning to obtain the feature representations of the essential components of traffic data without labelled data. In the second stage, we use improved self-knowledge distillation to transfer the feature representations of network traffic learned by the large convolutional neural network to the depthwise separable convolutional network. With these two stages, we can then obtain a lightweight, real-time and unsupervised IoT intrusion detection model. We then apply the CL-SKD model to the IoT datasets KDD CUP99, NSL-KDD, UNSW-NB15, CIC IDS2017 as well as BoT-IoT for binary and multi classification tasks, it can be concluded that our model performs better than the state-of-the-art models according to the experimental results.

Although the CL-SKD model can achieve excellent network traffic classification results, we admit that the CL-SKD model contains a lot of hyperparameters such as batch size, learning rate, the k , the memory queue length and so on, manual selection of hyperparameters is a time-consuming and labour-intensive task, feature work can use hyperparameter optimization algorithm for hyperparameter tuning. In addition, with the help of model quantization, network pruning and feature selection techniques, we can further reduce the model's number of parameters and computational cost to obtain a more lightweight intrusion detection model. Lastly, given our current research conditions and experimental setup, we recognize that we currently lack the capability to assess the statistical significance of performance differences among classifiers. We acknowledge this limitation and plan to address it in our future research.

CRedit authorship contribution statement

Zeyu Li: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing – original draft, Visualization. **Wenbin Yao:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Alaba, F. A., Othman, M., Hashem, I. A. T., & Alotaibi, F. (2017). Internet of Things security: A survey. *Journal of Network and Computer Applications*, 88, 10–28.
- Abdel-Basset, M., Chang, V., Hawash, H., Chakraborty, R. K., & Ryan, M. (2020). Deep-IFS: Intrusion detection approach for industrial internet of things traffic in fog environment. *IEEE Transactions on Industrial Informatics*, 17(11), 7704–7715.
- Abdel-Basset, M., Hawash, H., Chakraborty, R. K., & Ryan, M. J. (2021). Semi-supervised spatiotemporal deep learning for intrusions detection in IoT networks. *IEEE Internet of Things Journal*, 8(15), 12251–12265.
- Agarwal, R., & Joshi, M. V. (2001, April). PNRule: a new framework for learning classifier models in data mining (a case-study in network intrusion detection). In *Proceedings of the 2001 SIAM International Conference on Data Mining* (pp. 1–17). Society for Industrial and Applied Mathematics.
- Aghdam, Z. N., Rahmani, A. M., & Hosseinzadeh, M. (2021). The role of the Internet of Things in healthcare: Future trends and challenges. *Computer Methods and Programs in Biomedicine*, 199, Article 105903.
- Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., & Joulin, A. (2021). Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 9650–9660).
- Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020, November). A simple framework for contrastive learning of visual representations. In *International conference on machine learning* (pp. 1597–1607). PMLR.
- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys (CSUR)*, 41(3), 1–58.
- Dian, F. J., Vahidnia, R., & Rahmati, A. (2020). Wearables and the Internet of Things (IoT), applications, opportunities, and challenges: A Survey. *IEEE Access*, 8, 69200–69211.
- Ding, H., Chen, L., Dong, L., Fu, Z., & Cui, X. (2022). Imbalanced data classification: A KNN and generative adversarial networks-based hybrid approach for intrusion detection. *Future Generation Computer Systems*, 131, 240–254.
- Da Costa, K. A., Papa, J. P., Lisboa, C. O., Munoz, R., & de Albuquerque, V. H. C. (2019). Internet of Things: A survey on machine learning-based intrusion detection approaches. *Computer Networks*, 151, 147–157.
- Dhanabal, L., & Shantharajah, S. P. (2015). A study on NSL-KDD dataset for intrusion detection system based on classification algorithms. *International Journal of Advanced Research in Computer and Communication Engineering*, 4(6), 446–452.

- dos Santos, R. R., Viegas, E. K., Santin, A. O., & Cogo, V. V. (2022). Reinforcement learning for intrusion detection: More model longness and fewer updates. *IEEE Transactions on Network and Service Management*.
- Fatani, A., Abd Elaziz, M., Dahou, A., Al-Qaness, M. A., & Lu, S. (2021). IoT intrusion detection system using deep learning and enhanced transient search optimization. *IEEE Access*, 9, 123448–123464.
- Farahani, G. (2020). Feature selection based on cross-correlation for the intrusion detection system. *Security and Communication Networks*, 2020, 1–17.
- Fraihat, S., Makhadmeh, S., Awad, M., Al-Betar, M. A., & Al-Redhaei, A. (2023). Intrusion detection system for large-scale IoT NetFlow networks using machine learning with modified Arithmetic Optimization Algorithm. *Internet of Things*, 100819.
- Fang, Z., Wang, J., Wang, L., Zhang, L., Yang, Y., & Liu, Z. (2021). Seed: Self-supervised distillation for visual representation. arXiv preprint arXiv:2101.04731.
- Guezaz, A., Benkirane, S., Azrou, M., & Khurram, S. (2021). A reliable network intrusion detection approach using decision tree with enhanced data quality. *Security and Communication Networks*, 2021, 1–8.
- Gad, A. R., Nashat, A. A., & Barkat, T. M. (2021). Intrusion detection system using machine learning for vehicular ad hoc networks based on ToN-IoT dataset. *IEEE Access*, 9, 142206–142217.
- Gu, J., & Lu, S. (2021). An effective intrusion detection approach using SVM with naïve Bayes feature embedding. *Computers & Security*, 103, Article 102158.
- Grill, J. B., Strub, F., Alché, F., Tallec, C., Richemond, P., Buchatskaya, E., & Valko, M. (2020). Bootstrap your own latent-a new approach to self-supervised learning. *Advances in Neural Information Processing Systems*, 33, 21271–21284.
- Ge, J. W., Lu, D. B., & Fang, Y. Q. (2010, July). A revised training mechanism for AdaBoost algorithm. In *2010 IEEE International Conference on Software Engineering and Service Sciences* (pp. 491–494). IEEE.
- Gaber, T., Awotunde, J. B., Polorunso, S. O., Ajagbe, S. A., & Eldesouky, E. (2023). Industrial internet of things intrusion detection method using machine learning and optimization techniques. *Wireless Communications and Mobile Computing*, 2023(1), 3939895.
- Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531.
- He, K., Fan, H., Wu, Y., Xie, S., & Girshick, R. (2020). Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 9729–9738).
- Jan, S. U., Ahmed, S., Shakhov, V., & Koo, I. (2019). Toward a lightweight intrusion detection system for the internet of things. *IEEE Access*, 7, 42450–42471.
- Koroniotis, N., Moustafa, N., Sitnikova, E., & Turnbull, B. (2019). Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Future Generation Computer Systems*, 100, 779–796.
- Khanday, S. A., Fatima, H., & Rakesh, N. (2023). Implementation of intrusion detection model for DDoS attacks in Lightweight IoT Networks. *Expert Systems with Applications*, 215, Article 119330.
- Kareem, S. S., Mostafa, R. R., Hashim, F. A., & El-Bakry, H. M. (2022). An effective feature selection model using hybrid metaheuristic algorithms for iot intrusion detection. *Sensors*, 22(4), 1396.
- Koohpayegani, S. A., Tejankar, A., & Pirsiavash, H. (2021). Mean shift for self-supervised learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 10326–10335).
- Kim, T. H., Ramos, C., & Mohammed, S. (2017). Smart city and IoT. *Future Generation Computer Systems*, 76, 159–162.
- Li, H. (2018). Deep learning for natural language processing: Advantages and challenges. *National Science Review*, 5(1), 24–26.
- Lu, H., Wang, T., Xu, X., & Wang, T. (2021). Cognitive memory-guided autoencoder for effective intrusion detection in internet of things. *IEEE Transactions on Industrial Informatics*, 18(5), 3358–3366.
- Lazzarini, R., Tianfield, H., & Charissis, V. (2023). A stacking ensemble of deep learning models for IoT intrusion detection. *Knowledge-Based Systems*, 279, Article 110941.
- Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M., & Portmann, M. (2022 April). In *EdgeSage: A graph neural network based intrusion detection system for iot* (pp. 1–9). IEEE.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- Moustafa, N., & Slay, J. (2015, November). UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 military communications and information systems conference (MILCIS)* (pp. 1–6). IEEE.
- Murali, S., & Jamalipour, A. (2019). A lightweight intrusion detection for sybil attack under mobile RPL in the internet of things. *IEEE Internet of Things Journal*, 7(1), 379–388.
- Nimbalkar, P., & Kshirsagar, D. (2021). Feature selection for intrusion detection system in Internet-of-Things (IoT). *ICT Express*, 7(2), 177–181.
- Roy, S., Li, J., Choi, B. J., & Bai, Y. (2022). A lightweight supervised intrusion detection mechanism for IoT networks. *Future Generation Computer Systems*, 127, 276–285.
- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp*, 1, 108–116.
- Safaldin, M., Otair, M., & Abualigah, L. (2021). Improved binary gray wolf optimizer and SVM for intrusion detection system in wireless sensor networks. *Journal of Ambient Intelligence and Humanized Computing*, 12, 1559–1576.
- Sharma, N. V., & Yadav, N. S. (2021). An optimal intrusion detection system using recursive feature elimination and ensemble of classifiers. *Microprocessors and Microsystems*, 85, Article 104293.
- Sahu, S. K., Mohapatra, D. P., Rout, J. K., Sahoo, K. S., Pham, Q. V., & Dao, N. N. (2022). A LSTM-FCNN based multi-class intrusion detection using scalable framework. *Computers and Electrical Engineering*, 99, Article 107720.
- Singh, S., & Gupta, P. (2014). Comparative study ID3, cart and C4. 5 decision tree algorithm: A survey. *International Journal of Advanced Information Science and Technology (IJAIST)*, 27(27), 97–103.
- Telikani, A., Rudbardeh, N. E., Soleymanpour, S., Shahbahrani, A., Shen, J., Gaydadjiev, G., & Hassanpour, R. (2023). A cost-sensitive machine learning model with multitask learning for intrusion detection in IoT. *IEEE Transactions on Industrial Informatics*.
- Telikani, A., Shen, J., Yang, J., & Wang, P. (2022). Industrial IoT intrusion detection via evolutionary cost-sensitive learning and fog computing. *IEEE Internet of Things Journal*, 9(22), 23260–23271.
- Tavallae, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009 July). In *A detailed analysis of the KDD CUP 99 data set* (pp. 1–6). IEEE.
- Voulodimos, A., Doulamis, N., Doulamis, A., & Protopapadakis, E. (2018). Deep learning for computer vision: A brief review. *Computational Intelligence and Neuroscience*, 2018.
- Wu, P., & Guo, H. (2019 December). In *LuNET: a deep neural network for network intrusion detection* (pp. 617–624). IEEE.
- Wang, Z., Liu, Y., He, D., & Chan, S. (2021). Intrusion detection methods based on integrated deep learning model. *Computers & Security*, 103, Article 102177.
- Wang, Z., Li, Z., Wang, J., & Li, D. (2021). Network intrusion detection model based on improved BYOL self-supervised learning. *Security and Communication Networks*, 2021, 1–23.
- Wang, Z., Li, Z., He, D., & Chan, S. (2022). A lightweight approach for network intrusion detection in industrial cyber-physical systems based on knowledge distillation and deep metric learning. *Expert Systems with Applications*, 206, Article 117671.
- Wang, Z., Xu, Z., He, D., & Chan, S. (2021). Deep logarithmic neural network for Internet intrusion detection. *Soft Computing*, 25(15), 10129–10152.
- Zielonka, A., Sikora, A., Woźniak, M., Wei, W., Ke, Q., & Bai, Z. (2020). Intelligent internet of things system for smart home optimal convection. *IEEE Transactions on Industrial Informatics*, 17(6), 4308–4317.
- Zhao, R., Gui, G., Xue, Z., Yin, J., Ohtsuki, T., Adebisi, B., & Gacanin, H. (2021). A novel intrusion detection method based on lightweight neural network for internet of things. *IEEE Internet of Things Journal*, 9(12), 9960–9972.
- Zhou, X., Liang, W., Li, W., Yan, K., Shimizu, S., Kevin, I., & Wang, K. (2021). Hierarchical adversarial attacks against graph-neural-network-based IoT network intrusion detection system. *IEEE Internet of Things Journal*, 9(12), 9310–9319.
- Zhang, Z., Geiger, J., Pohjalainen, J., Mousa, A. E. D., Jin, W., & Schuller, B. (2018). Deep learning for environmentally robust speech recognition: An overview of recent developments. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 9(5), 1–28.